

# Parte II

## Modulo B

|                 |                                                   |            |
|-----------------|---------------------------------------------------|------------|
| <b>8</b>        | <b>SAP-3</b>                                      | <b>117</b> |
| 8.1             | Istruzioni . . . . .                              | 117        |
| 8.1.1           | Istruzioni di trasferimento dati . . . . .        | 118        |
| 8.1.2           | Istruzioni aritmetiche . . . . .                  | 118        |
| 8.1.3           | Istruzioni di rotazione . . . . .                 | 120        |
| 8.1.4           | Istruzioni logiche . . . . .                      | 121        |
| 8.1.5           | Istruzioni jump . . . . .                         | 122        |
| 8.1.6           | Istruzioni extended-register . . . . .            | 123        |
| 8.1.7           | Istruzioni indirette . . . . .                    | 124        |
| 8.1.8           | Istruzioni stack . . . . .                        | 125        |
| 8.1.9           | Tabella complessiva . . . . .                     | 128        |
| <b>Esercizi</b> |                                                   | <b>137</b> |
|                 | Circuiti sequenziali . . . . .                    | 137        |
|                 | Aritmetica nei calcolatori . . . . .              | 146        |
|                 | SAP-1 . . . . .                                   | 152        |
|                 | SAP-2 . . . . .                                   | 155        |
|                 | SAP-3 . . . . .                                   | 159        |
| <b>II</b>       | <b>Modulo B</b>                                   | <b>163</b> |
| <b>9</b>        | <b>Data management</b>                            | <b>165</b> |
| 9.1             | Dai dati ai dataset . . . . .                     | 166        |
| 9.1.1           | Metadati . . . . .                                | 167        |
| 9.1.2           | Strutture di dati . . . . .                       | 168        |
| 9.2             | Archiviazione . . . . .                           | 171        |
| 9.2.1           | Dispositivi di memoria . . . . .                  | 171        |
| 9.2.2           | Legge di Kryder . . . . .                         | 175        |
| 9.2.3           | Scalabilità . . . . .                             | 176        |
| 9.3             | Affidabilità e sicurezza . . . . .                | 178        |
| 9.3.1           | Strategie di affidabilità . . . . .               | 179        |
| 9.3.2           | Crittografia . . . . .                            | 183        |
| 9.3.3           | Autenticazione e autorizzazione . . . . .         | 185        |
| 9.3.4           | Codifica e decodifica dell'informazione . . . . . | 185        |
| 9.3.5           | Funzioni hash . . . . .                           | 187        |
| 9.4             | File System . . . . .                             | 188        |
| 9.4.1           | Caratteristiche . . . . .                         | 189        |
| 9.4.2           | Allocazione dello spazio . . . . .                | 189        |
| 9.4.3           | Limiti . . . . .                                  | 192        |
| 9.5             | File system distribuito . . . . .                 | 193        |
| 9.5.1           | Funzionamento . . . . .                           | 194        |
| 9.5.2           | Esempio: HDFS . . . . .                           | 195        |
| 9.5.3           | Teorema CAP . . . . .                             | 197        |
| 9.6             | Database . . . . .                                | 200        |

|          |                                              |            |
|----------|----------------------------------------------|------------|
| 9.6.1    | Modelli di database . . . . .                | 201        |
| 9.6.2    | Database relazionali . . . . .               | 202        |
| 9.6.3    | SQL . . . . .                                | 204        |
| 9.7      | Database NoSQL . . . . .                     | 217        |
| 9.7.1    | Partizione dei database . . . . .            | 218        |
| 9.8      | Caratteristiche dei database NoSQL . . . . . | 220        |
| 9.9      | Esempi di database NoSQL . . . . .           | 221        |
| 9.9.1    | Database key-value . . . . .                 | 221        |
| 9.9.2    | Database document . . . . .                  | 222        |
| 9.9.3    | Database time series . . . . .               | 222        |
| 9.9.4    | FullText Search Engine . . . . .             | 222        |
| <b>A</b> | <b>Tabella ASCII</b>                         | <b>223</b> |
| <b>B</b> | <b>Tabellina del 16</b>                      | <b>224</b> |
| <b>C</b> | <b>Calcolo del throughput nel RAID 0</b>     | <b>225</b> |

# Capitolo 9

## Data management

Di seguito discuteremo dei vari step che conducono dall'acquisizione dati tramite sensori alla produzione di vera e propria *informazione*. Il processo può essere suddiviso in due parti:

1. **Sistema di acquisizione dati**, suddiviso nelle seguenti parti:

- **Sensori**, utilizzati per convertire grandezze fisiche in segnali elettrici.
- **Elettronica frontend**, con il ruolo di interfaccia tra i sensori e il sistema di elaborazione del segnale; quest'ultimo viene infatti amplificato e filtrato in modo da rimuovere rumore indesiderato.
- **Elettronica readout**, responsabile della conversione digitale del segnale analogico; può anche svolgere la funzione di memoria temporanea del segnale.
- **Trigger & DAQ**: il primo determina il momento in cui il sistema può acquisire dati, ad esempio dopo il superamento di una certa soglia in ingresso o dopo un certo intervallo temporale; il secondo filtra ulteriormente il segnale (ora digitalizzato) e lo memorizza.

2. **Modello di calcolo**, diviso nei seguenti aspetti:

- **Archiviazione**, ovvero il modo e il posto in cui fisicamente vengono memorizzati i dati ottenuti dal sistema di acquisizione.
- **Affidabilità e sicurezza**, ovvero le tecniche adottate per preservare i dati e i permessi per accedervi.
- **Querying e filtraggio**, ovvero il processo di ricerca e selezione dei dati; il *querying* è quel processo per cui è possibile estrarre dati dalla memoria in funzione di determinati criteri.
- **Elaborazione**, che coinvolge l'applicazione di algoritmi e strumenti di calcolo per ricavare informazioni utili a partire dai dati memorizzati.



Si tenga presente che quello presentato è un sistema di acquisizione di dati *fisici*; più in generale la sorgente di dati può essere di tipo molto diverso: è possibile lavorare con dati *simulati* (si pensi alle varie tecniche Monte Carlo), con documenti di testo, con immagini e così via.

## 9.1 Dai dati ai dataset

Per **dataset** si intende un insieme di *informazioni* (dati o simulazioni) raccolte da una *serie di fonti* (rivelatori e dispositivi in generale) in un certo *arco di tempo* associate a un unico *ambito* o a un unico *scopo*.

Il dato, ovvero l'unità base dell'informazione contenuta in un dataset, viene spesso indicato come *datum* o anche come *record*.

Di seguito è illustrata la formazione di un dataset:

1. Un sistema di acquisizione dati (**DAQ**) scrive costantemente in memoria ogni informazione che viene raccolta dai sensori alla più alta frequenza possibile.
2. I dati memorizzati vengono poi sottoposti a una prima elaborazione, mediamente dopo un certo numero di ore o di giorni, che include la pulizia, la correzione di eventuali errori, la normalizzazione e - se necessario - anche la riduzione della dimensionalità.
3. In un arco di tempo più lungo (mediamente ogni anno) i dati vengono sottoposti a un'ulteriore elaborazione che coinvolge tecniche più sofisticate, come ad esempio simulazioni Monte Carlo.
4. I dati che vengono processati in qualunque arco di tempo sono archiviati a lungo termine in uno spazio di memoria dedicato.

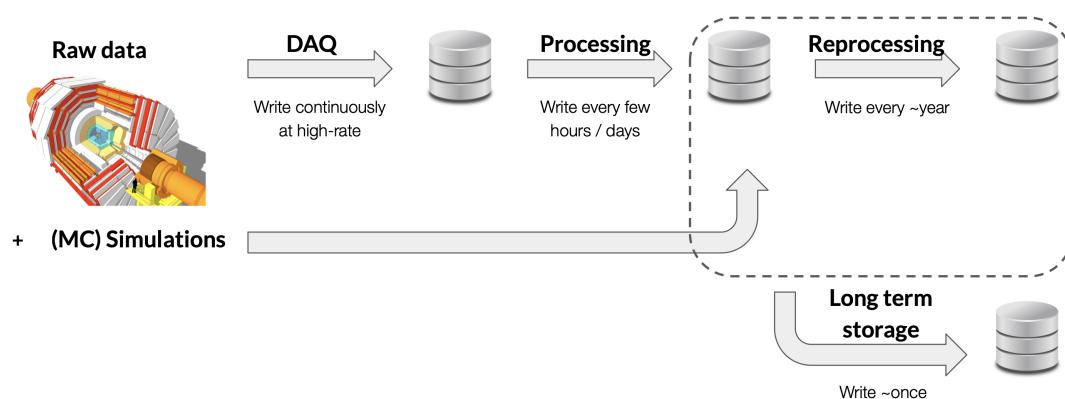


Figura 9.1: Formazione di un dataset

Le sequenze di dati acquisite vengono solitamente suddivise in base allo scopo della collezione: la suddivisione può essere banalmente suddivisa per archi di

tempo (ad esempio per anno) o per campagne di acquisizione dati (si pensi alle varie *Run* che vengono eseguite al LHC); un'altra possibilità è la suddivisione per versione del software utilizzato, scelta adottata principalmente per modelli di dati simulati.

### 9.1.1 Metadati

I dataset sono spesso accompagnati da **metadati**, così chiamati perché sono dati la cui funzione è quella di dare informazione riguardo i dati stessi contenuti nel dataset.

Essi svolgono tre funzioni principali:

| Descrittiva                       | Strutturale                               | Amministrativa                    |
|-----------------------------------|-------------------------------------------|-----------------------------------|
| <i>Che cosa significa?</i>        | <i>Come si rapporta ad altri dataset?</i> | <i>Quando è stato creato?</i>     |
| <i>A chi appartiene?</i>          | <i>Come è organizzato?</i>                | <i>Quando è stato modificato?</i> |
| <i>Che cosa contiene?</i>         | <i>In funzione di cosa è suddiviso?</i>   | <i>Chi può avere accesso?</i>     |
| <i>Quando è stato pubblicato?</i> | <i>Come è ordinato?</i>                   | <i>Quanto è grande?</i>           |

Per comprendere l'importanza dei metadati guardiamo a due file molto diversi; il primo è il seguente:



*a\_nice\_cat\_photo.jpeg*

È evidente che in questo caso i metadati possono risultare superflui: un qualunque programma di visualizzazione di immagini mostrerà la generica immagine di un gatto; i metadati potrebbero comunque fornire informazioni ulteriori, come ad esempio chi ha scattato la foto, in che cartella si trova, qual è il nome del gatto, e così via.

Passiamo ora all'altro file:

```

...
0291ce0 3235 3631 3020 3030 3030 6e20 0a20 3030
0291cf0 3230 3836 3235 3933 3020 3030 3030 6e20
0291d00 0a20 3030 3230 3836 3235 3036 3020 3030
0291d10 3030 6e20 0a20 3030 3230 3836 3235 3338
0291d20 3020 3030 3030 6e20 0a20 3030 3230 3836
0291d30 3335 3430 3020 3030 3030 6e20 0a20 3030
0291d40 3230 3836 3335 3632 3020 3030 3030 6e20
0291d50 0a20 3030 3230 3836 3335 3734 3020 3030
0291d60 3030 6e20 0a20 3030 3230 3836 3335 3936
0291d70 3020 3030 3030 6e20 0a20 3030 3230 3836
0291d80 3335 3039 3020 3030 3030 6e20 0a20 3030
0291d90 3230 3836 3435 3331 3020 3030 3030 6e20
...

```

*3CBC07C2-4B64-E611-B423-02163E0124E9.root*

Senza alcun metadato è impossibile risalire al senso di quanto contenuto nel file: si tratta, infatti, di dati di collisione protone-protone a  $13\text{ TeV}$  avvenuti nel CMS, parte di un dataset collezionato nel 2018 facente parte della campagna *Run-D* grande  $3\text{ GB}$  a cui hanno accesso solo gli utenti dell'esperimento CMS. Niente di tutto questo sarebbe stato pensabile senza dei dati che ci dicessero esplicitamente quanto appena scritto.

### 9.1.2 Strutture di dati

I dataset possono essere suddivisi in tre diversi tipi in funzione del modo in cui sono ordinati:

#### 1. **Dati strutturati**

Sono organizzati in un formato tabellare *rigido*, in modo che ogni elemento sia “incasellato” in una posizione univoca determinata da una riga e da una colonna.

Nella figura a seguire possiamo osservare un esempio di dati strutturati: sulla sinistra è mostrato il formato *raw*, ovvero i veri e propri bit conservati all'interno della memoria (in forma esadecimale per questione di comodità e compattezza), mentre sulla destra gli stessi dati sono in un formato di testo per noi leggibile, ovvero numeri decimali separati da virgole e spazi.

|      |      |      |      |                              |
|------|------|------|------|------------------------------|
| e86d | 95d2 | 0512 | 407a | 1, 0, 61, 42552041, 1859, 13 |
| 4290 | 95d4 | 0512 | 404a | 1, 0, 37, 42552042, 532, 16  |
| 462b | 95ed | 0512 | 40c0 | 1, 0, 96, 42552054, 2609, 11 |
| 10db | 95fa | 0512 | 4020 | 1, 0, 16, 42552061, 134, 27  |
| 8923 | 95fa | 0512 | 40f0 | 1, 0, 120, 42552061, 1097, 3 |
| c95a | 95fe | 0512 | 4068 | 1, 0, 52, 42552063, 1610, 26 |

Figura 9.2: Esempio di dati strutturati

La forma tabellare può essere visualizzata utilizzando diverse opzioni, tra cui quella standard offerta dalla libreria *pandas* di Python.

| HEAD | FPGA | TDC_CHANNEL | ORBIT_CNT | BX_COUNTER | TDC_MEAS |
|------|------|-------------|-----------|------------|----------|
| 1    | 0    | 61          | 42552041  | 1859       | 13       |
| 1    | 0    | 37          | 42552042  | 532        | 16       |
| 1    | 0    | 96          | 42552054  | 2609       | 11       |
| 1    | 0    | 16          | 42552061  | 134        | 27       |
| 1    | 0    | 120         | 42552061  | 1097       | 3        |
| 1    | 0    | 52          | 42552063  | 1610       | 26       |

Figura 9.3: Visualizzazione tramite *pandas* dei dati mostrati nella precedente figura

Esempi di dati strutturati sono i file in formato *.csv* (*comma-separated values*) e gli *RDBMS* (*Relational DataBase Management System*), chiamati anche *database relazionali* adatti al linguaggio SQL.

Il vantaggio principale dei dati strutturati è offerto proprio dalla loro struttura, essendo già “impacchettati” secondo un ordine chiaro e stabilito. Lo svantaggio è dato invece dalla rigidità di tale schema, che limita la possibilità di aggiungere nuove colonne e di agire su eventuali errori nel posizionamento (si pensi, ad esempio, all’inversione tra due elementi nella stessa riga).

## 2. Dati semi-strutturati

Sono organizzati secondo logiche strutturate ma non possiedono uno schema fissato a priori, essendo deducibile dai dati stessi attraverso *tag* e *markup*.

Esempi di dati semi-strutturati sono i file in formato *.xml* (*extensible markup language*), i file in formato *.json* (*JavaScript Object Notation*) e i file in formato *.yaml* (*YAML ain’t markup language*), questi ultimi usati per le email.

```

<person>
  <firstName>John</firstName>
  <lastName>Smith</lastName>
  <age>25</age>
  <address>
    <streetAddress>21 2nd Street</streetAddress>
    <city>New York</city>
    <state>NY</state>
    <postalCode>10021</postalCode>
  </address>
  <phoneNumbers>
    <phoneNumber>
      <type>home</type>
      <number>212 555-1234</number>
    </phoneNumber>
  </phoneNumbers>
  <sex>
    <type>male</type>
  </sex>
</person>

```

```

<person>
  <firstName>John</firstName>
  <lastName>Smith</lastName>
  <age>25</age>
  <address>
    <streetAddress>21 2nd Street</streetAddress>
    <city>New York</city>
    <state>NY</state>
    <postalCode>10021</postalCode>
  </address>
  <phoneNumbers>
    <phoneNumber>
      <type>home</type>
      <number>212 555-1234</number>
    </phoneNumber>
  </phoneNumbers>
  <sex>
    <type>male</type>
  </sex>
</person>

```

Formato .xml
Formato .json

Figura 9.4: Esempio di dati semi-strutturati in due formati diversi

Per verificare l'integrità dei dati è sempre possibile convertire dei dati semi-strutturati in dati strutturati.

| First name | Last name | Age | Address | Phone numbers | Sex |
|------------|-----------|-----|---------|---------------|-----|
| John       | Smith     | 25  |         |               | M   |

| Street address | City     | State | Postal code |
|----------------|----------|-------|-------------|
| 21 2nd Street  | New York | NY    | 10021       |

| Type | Number       |
|------|--------------|
| Home | 212 555-1234 |

Figura 9.5: Dati semi-strutturati illustrati nella precedente figura convertiti in dati strutturati

### 3. Dati non strutturati

Non sono schematizzati tramite alcun modello né vi è alcuna struttura suggerita dai dati stessi.

Si tratta solitamente di dati che definiscono un *flusso continuo* di informazione, come ad esempio file di naturale testuale, audio, video e immagini.

Il vantaggio principale di questo tipo di formato è la massima flessibilità, adattandosi bene a qualunque tipo di dato, mentre l'altra faccia della medaglia è il lavoro difficile e laborioso per estrarre singoli pezzi di informazione dal dataset.

## 9.2 Archiviazione

In figura 9.1 abbiamo avuto modo di osservare come ogni unità di archiviazione svolgesse un ruolo diverso, seppur lo scopo fosse sempre quello di mantenere in memoria i dati ricevuti in ingresso:

- Il DAQ scrive in una memoria a frequenze molto elevate, compiendo dunque svariati cicli di scrittura in poco tempo; tali dati, però, non resteranno a lungo in quella unità di memoria: questi verranno processati e mandati altrove, liberando così dello spazio che può essere riempito da nuova informazione proveniente dal sistema di acquisizione dati.
- I dati processati sono distribuiti in due diverse unità: in entrambi i casi la scrittura avviene in archi di tempo molto più ampi, e al contempo la quantità di memoria utilizzata cresce considerevolmente.
- Gli stessi dati processati vengono copiati in un'ulteriore memoria, che deve dunque essere di ingenti dimensioni ma dalla quale non sono previsti poi tanti cicli di lettura.

Notiamo quindi delle *esigenze diverse* relativamente alle unità di memoria: a un decrescente numero di operazioni di lettura e scrittura corrisponde una crescente quantità di spazio a disposizione. Vedremo di seguito come esigenze diverse si tradurranno in *tecnologie diverse* adatte a soddisfarle.

Queste tecnologie si distinguono per diversi aspetti, tutti legati alle richieste di input/output (I/O) che possono essere lettura, scrittura, aggiornamento e cancellazione. Tali aspetti sono elencati di seguito:

- **Latenza**: tempo impiegato per eseguire una singola I/O
- **Frequenza**: numero di I/O eseguite al secondo (**IOPS**, ovvero *I/O per second*)
- **Block size**: dimensione in byte di una singola I/O (ogni operazione, infatti, coinvolge un certo numero di bit fissato)
- **Throughput**: prodotto tra *block size* e *frequenza*, ovvero quantità di bit trasferiti in un certo intervallo di tempo, misurata convenzionalmente in *MB/s*

### 9.2.1 Dispositivi di memoria

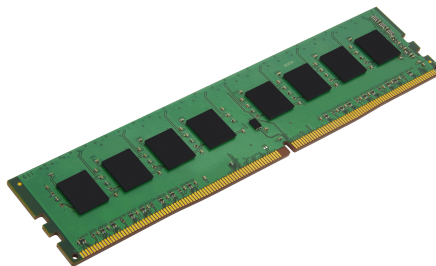
I vari dispositivi che vedremo saranno elencati da quello con la latenza più bassa a quello con la latenza più alta.

### DRAM (Dynamic Random Access Memory)

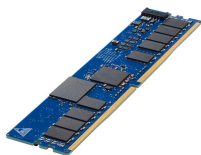
Immagazzina i bit di dati in delle **celle NAND**, composte da un transistor collegato alla linea di bus accoppiato con un condensatore.

Come sappiamo, il termine *RAM* sta a indicare che è possibile accedere ai dati in qualunque posizione essi si trovino alla stessa velocità; il termine *dynamic* all'inizio, invece, specifica che i dati non restano in memoria in modo persistente, dato che quando viene interrotta l'alimentazione del dispositivo tutti i dati vengono persi.

Il suo difetto principale risiede proprio in quest'ultima caratteristica, che la rende un *temporary storage*; al contempo, il suo pregio principale è la rapidità nell'esecuzione delle operazioni I/O, avendo una latenza  $\sim 0.1 \mu s$ . Vengono utilizzati per processi che richiedono una frequenza di scrittura molto alta.



### NVM (Non-Volatile Memory)



Un'alternativa alla DRAM che ha una latenza tutto sommato simile ( $\sim 1 \mu s$ ) ma la caratteristica di mantenere i dati salvati anche quando viene a mancare l'alimentazione del dispositivo. Queste qualità richiedono, però, un costo molto alto: per questa ragione dispositivi di questo tipo (come anche la DRAM) non sono utilizzati in applicazioni standard, ma vengono impiegati in progetti specifici che richiedono dispositivi con queste caratteristiche.

### SSD (Solid State Drives)

A causa del loro deprezzamento nel corso del tempo sono diventati diffusi anche per il grande pubblico, essendo inclusi all'interno di comuni personal computer.

La novità sta nel modo in cui viene immagazzinata l'informazione, che non è più situata in delle celle NAND ma in delle **multi-level charge trap**: si tratta di condensatori che non distinguono solo tra due livelli di carica (carico/scarico, corrispondenti agli stati 1/0) ma tra più livelli; nella maggior parte dei dispositivi questi livelli sono otto, potendo così codificare un totale di tre bit di informazione.

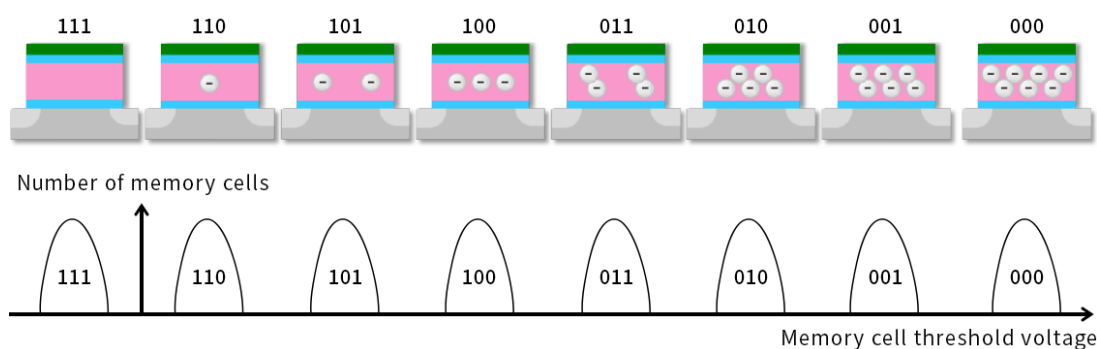


Figura 9.6: Tecnologia TLC: in alto la codifica dei vari stati di carica in tre bit; in basso i *threshold* di tensione che definiscono gli otto stati

Questa tecnologia, chiamata *TLC* (*Triple-level cell*),<sup>1</sup> permette di ampliare la *storage density*,<sup>2</sup> risultando in dispositivi molto più compatti dei precedenti a parità di memoria. Di contro, essa limita la capacità di scrittura: ogni volta che è necessario scrivere un nuovo pezzo di informazione è necessario eliminare interamente ciò che era prima contenuto nella cella, scaricando interamente la carica dal condensatore. Si tratta dunque di dispositivi con un'alta frequenza di lettura ( $\sim 10 \mu s$ ) ma con una frequenza di scrittura considerevolmente più bassa ( $\sim 100 \mu s$ ).

Un altro limite risiede nell'inevitabile processo di isteresi a cui vanno incontro le varie celle nei cicli di scrittura: a ogni ciclo corrisponde una piccola carica residua che non permette più il corretto funzionamento oltre un certo accumulo. Tale limite è noto e previsto fin dalla costruzione del dispositivo, infatti esso comprende anche un *controller* che memorizza a quanti cicli di scrittura è stata sottoposta ciascuna cella di memoria: superato il numero massimo previsto viene impedita la memorizzazione in quelle locazioni. Ne segue che ogni SSD ha una "scadenza" prevista fin dal principio, tanto più vicina quanto più frequente è l'utilizzo: si adatta dunque bene alle esigenze di un utente medio, ma può risultare inadatto in altri contesti come quelli di ricerca.

## HDD (Hard Disks Drives)

Sono i dispositivi di memoria tutt'oggi più diffusi, essendo un ottimo compromesso in termini di costo relativamente alle loro prestazioni.

È formato da dischi magnetizzati su ambo i lati i cui domini sono distribuiti lungo cerchi concentrici: i singoli domini magnetici sono chiamati *track*, i quali appartengono a dei *sector* a loro volta distribuiti sul lato di un disco, chiamato *platter*. Al contrario di quel che si potrebbe pensare, i bit non sono associati ciascuno a una track ma a due track consecutive, indicando il bit 0 con due track con stessa polarizzazione e il bit 1 con due track con polarizzazione opposta.

<sup>1</sup>In generale è indicata con *MLC*, ovvero *Multi-level cell*.

<sup>2</sup>Quantità di dati memorizzati per unità di superficie.



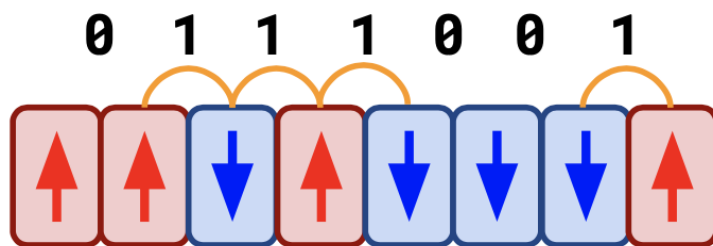
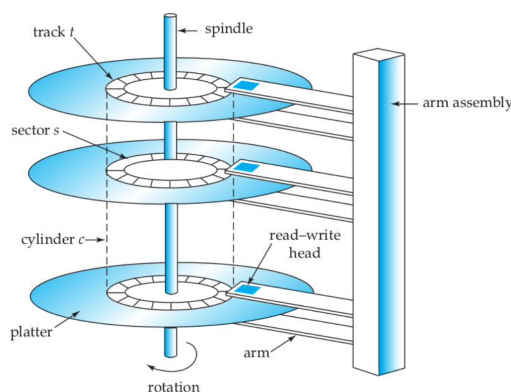


Figura 9.7: Codificazione dei bit in funzione della variazione della polarizzazione tra due track contigue

La lettura dei dati non avviene per track ma per sector, che contengono solitamente dagli 0.5 ai 4  $kB$ , dunque anche la scrittura su disco è spezzata secondo questa quantità di dati che rappresenta la block size.

La lettura e scrittura avviene attraverso dei bracci meccanici, che leggono i vari sector esattamente come fa una testina col giradischi, con la capacità di allontanarsi e avvicinarsi rispetto al centro del platter. I dischi, inoltre, hanno la capacità di ruotare a una certa frequenza.



I limiti nella velocità di lettura e scrittura dipendono proprio dalla natura meccanica del dispositivo. La latenza ( $\sim ms$ ), infatti, si può suddividere in due contributi:

- **Seek time:** tempo impiegato ad allineare il braccio alla track
- **Rotation delay:** tempo impiegato dal disco per ruotare allineandosi al sector

### Nastri magnetici

È la stessa tecnologia usata per le VHS e le cassette in generale: la logica è la stessa dei domini magnetici per gli HDD, con la differenza che in questo caso sono uno di seguito all'altro e il supporto fisico (ovvero il nastro) può essere avvolto in una forma compatta. È pensato per quei dati che vengono salvati con l'idea che non verranno più ripresi se non raramente e dopo tanto tempo, infatti tutti i grandi progetti scientifici hanno una copia di tutti i dati presi in forma di nastri all'interno di un magazzino, come ad esempio il CERN che a oggi è arrivato a contenere quasi 1  $EB$  ( $10^3 TB$ ) sottoforma di nastri.

La lettura di tali nastri nei magazzini avviene attraverso dei bracci robotici, che - nel momento in cui viene fornita l'istruzione da remoto - sono incaricati di

prelevare il nastro richiesto, portarlo al lettore che lo srotolerà fino al punto in cui avrà inizio la lettura. È facile capire, dunque, che la latenza non ha un vero e proprio valore fissato: dipende dalla distanza tra il braccio robotico e il nastro richiesto, dalla distanza tra il nastro e il lettore, ma anche dalla possibilità che tutti i bracci siano in quel momento occupati e serva attendere che se ne liberi qualcuno; in generale diremo che l'ordine di grandezza è  $\sim 100$  s, per quanto siamo consapevoli che il range di possibilità è molto ampio a causa dell'alta aleatorietà.

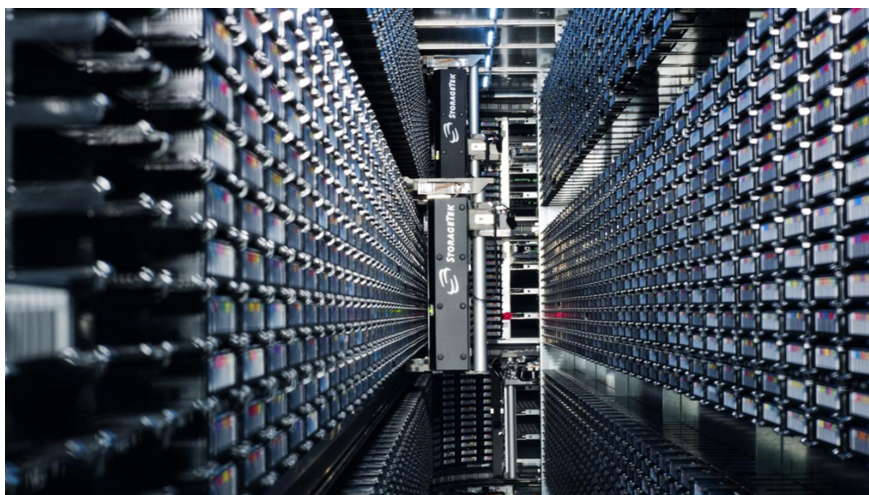


Figura 9.8: Libreria di nastri magnetici al CERN

È importante sottolineare che la lettura in sé e per sé è molto rapida, e la latenza così alta è imputabile esclusivamente agli aspetti sopra descritti.

I vantaggi di questo formato sono sostanzialmente due: il primo è il suo basso costo di produzione (che si presta all'ingente quantità di dati che deve contenere), e il secondo è la sua “robustezza”, dato che anche in casi estremi - come, ad esempio, un allagamento della struttura - i dati resterebbero intatti e non andrebbero persi.

### 9.2.2 Legge di Kryder

Un fattore da tenere in considerazione per l'archiviazione a lungo termine dei dati è il costante sviluppo delle tecnologie che operano in questo settore. È famosa, ad esempio, la *legge di Moore* (1965), secondo la quale il numero di transistor in un circuito integrato raddoppia ogni anno; nell'ambito della memorizzazione dei dati esiste una legge analoga, nota come **legge di Kryder**, che afferma che la storage density raddoppia ogni 13-14 mesi. Un altro effetto collaterale di quanto detto è il costante *deprezzamento* nel corso del tempo della stessa quantità di memoria, proprio a causa della riduzione della densità.

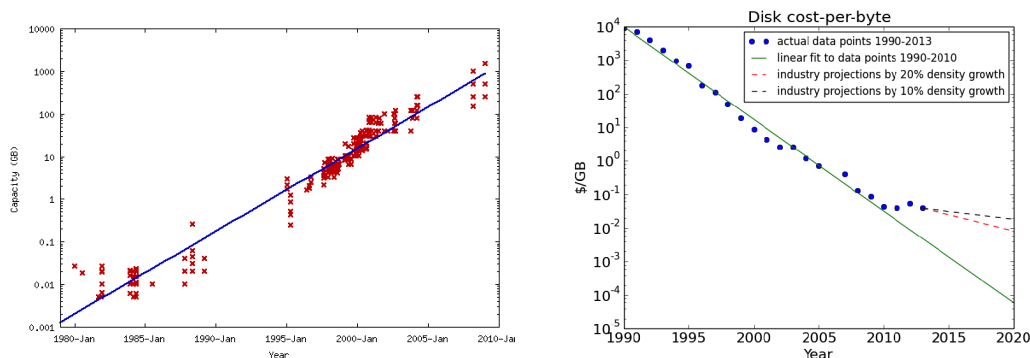


Figura 9.9: Legge di Kryder: a sinistra l'incremento della capacità, a destra il deprezzamento delle unità di memoria

Questa legge, proposta per la prima volta nel 1998 su basi empiriche, non è in realtà più valida da circa due decenni. Ciononostante resta valido il suo significato intrinseco: le tecnologie in quest'ambito evolvono molto rapidamente, ed è importante tenere in considerazione ciò nell'ottica di dataset che crescono nel corso degli anni.

### 9.2.3 Scalabilità

A questo punto ci ritroviamo ad affrontare nella pratica il problema: come gestiamo un dataset in costante crescita? Come cambieranno le performance (lettura e scrittura) conseguentemente all'aumento di memoria?

Questo problema ha un nome: *scalabilità*. Vedremo di seguito due filosofie diverse a riguardo.

- **Scalabilità verticale:** l'incremento di memoria viene eseguito sostituendo le risorse attuali con risorse più moderne e avanzate
- **Scalabilità orizzontale:** l'incremento di memoria viene eseguito collegando nuove risorse in catena, non necessariamente più performanti delle attuali, e formando un cosiddetto *sistema distribuito*

| Verticale                                                                     | Orizzontale                                                                                      |
|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Non vi è un reale cambiamento nel modo in cui l'utente accede alle risorse    | È possibile accrescere lo spazio in modo consistente con un prezzo relativamente basso           |
| Se le performance sono considerate soddisfacenti esse possono solo migliorare | La scalabilità è facilitata a livello hardware (basta il collegamento tramite cavi dei supporti) |

#### Risulta costoso

Esiste un limite alla quantità di memoria disponibile per un singolo dispositivo

Nel momento dell'aggiornamento è necessario interrompere le operazioni di salvataggio ed eseguire il trasferimento dei dati

Si affida alla connessione, dunque la larghezza di banda della rete può risultare limitante

Diventa laborioso accedere alle risorse essendo distribuiti su sistemi locali diversi

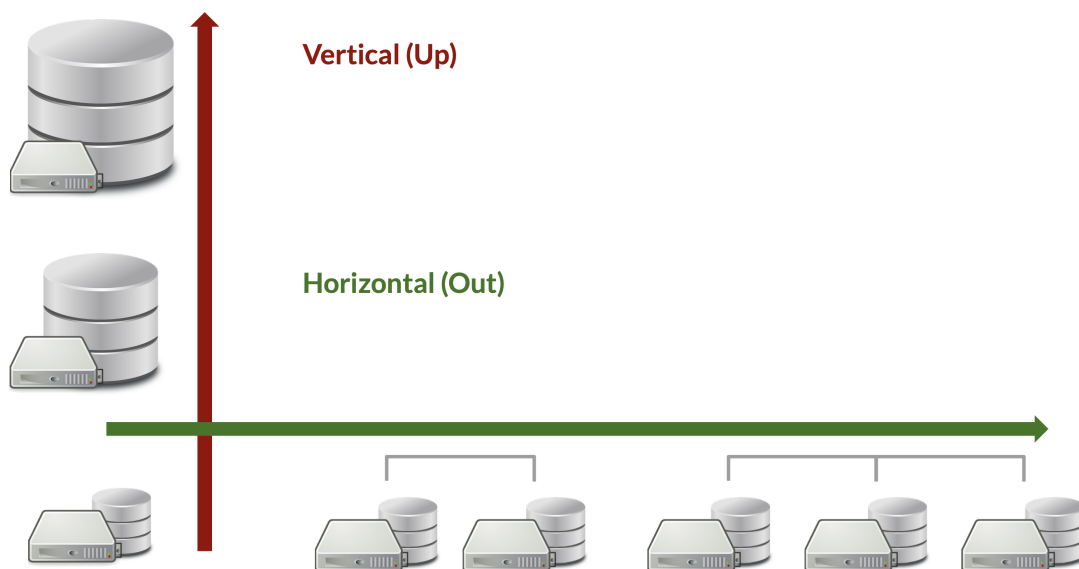


Figura 9.10: Le due possibili scalabilità a partire da un certo storage

### 9.3 Affidabilità e sicurezza

Finora abbiamo discusso riguardo *hardware*, ovvero supporti fisici che contengono i nostri dati. È importante sottolineare che si tratta, appunto, di *supporti*, ovvero di dispositivi il cui scopo è quello di conservare informazioni: purché i dati siano al sicuro - per quel che ci riguarda - essi possono danneggiarsi e rompersi anche irrimediabilmente. Riassumendo: quel che conta è l'integrità dei dati, non delle unità di memoria in sé.

Le minacce da cui dobbiamo proteggerci sono di tre tipi:

1. **Perdita dei dati:** se un dispositivo di memoria è soggetto a un guasto (sia esso di natura elettrica o meccanica), solitamente, tutti i dati contenuti al suo interno sono perduti
2. **Corruzione dei dati:** determinati fattori ambientali (abbassamento della tensione in ingresso, aumento della temperatura ambientale, radiazione cosmica, e così via) possono causare un *bit-flip*, ovvero l'inversione di un bit; per gli HDD, ad esempio, in media si ha un bit-flip ogni  $10^{14}$  bit coinvolti in cicli di lettura o scrittura
3. **Errori di trasferimento:** avvengono quando non vi sono adeguati check nella copia o trasferimento di dati

Da un punto di vista formale definiamo l'affidabilità come segue:

L'**affidabilità** corrisponde alla probabilità che un sistema continui a operare correttamente oltre un certo lasso di tempo o un certo numero di operazioni

In termini quantitativi definiamo l'**MTTF** (*Mean Time To Failure*), ovvero il tempo medio che intercorre tra la prima accensione del dispositivo e il momento in cui avviene un guasto. Per sistemi *reparabili* viene definito l'**MTBF** (*Mean Time Between Failure*), ovvero il tempo medio che intercorre tra un guasto e il successivo.



Figura 9.11: MTTF e MTBF in funzione del tempo

Realisticamente, l'MTTF di un HDD è pari a circa dieci anni; assumendo una probabilità di guasto equidistribuita in tutto questo arco di tempo troveremo la probabilità di guasto al giorno da un semplice rapporto:

$$p = \frac{1}{10 \cdot 365} = 0.00027$$

Questa è la probabilità associata a un singolo HDD. Considerando 5000 HDD, assumendo i dischi indipendenti, otteniamo

$$5000 \cdot 0.00027 = 1.35$$

Dunque, in media, si prevede che ogni giorno un HDD si guasterà, e di conseguenza dovrà essere sostituito. Se 5000 appare come un numero esageratamente alto, si pensi che al CERN si lavora quotidianamente con  $\sim 100\,000$  HDD, che si traduce in circa 27 unità di memoria danneggiate ogni giorno.

### 9.3.1 Strategie di affidabilità

Avendo ormai compreso che, in un modo o nell'altro, i dati sono costantemente a rischio all'interno dei dispositivi di memoria, è utile introdurre delle strategie allo scopo di conservarli in modo sicuro:

1. **Mirroring**: i dati vengono replicati su dispositivi diversi, a volte situati in posti diversi
2. **Striping**: i dati vengono suddivisi in “pezzi” e distribuiti su elementi di memoria diversi
3. **Checking**: vengono individuati (e, possibilmente, corretti) errori relativi alla coerenza dei dati all'interno dei dispositivi, come ad esempio una *checksum* o un *parity check*

La base su cui applicare queste strategie è la cosiddetta *disk redundancy*, indicata dalla sigla **RAID** (*Redundant Array of Independent Disks*), in cui più dispositivi vengono combinati per formare un un singolo dispositivo *logico* in modo da migliorare affidabilità e performance.

Prestiamo attenzione al fatto che il processo appena descritto non riguarda la scalabilità orizzontale, dato che qui i vari dispositivi vengono considerati come un'unica entità piuttosto che come elementi diversi che lavorano insieme per aumentare capacità e prestazioni.

Vedremo di seguito alcuni RAID che implementano le strategie elencate sopra.

## RAID 0

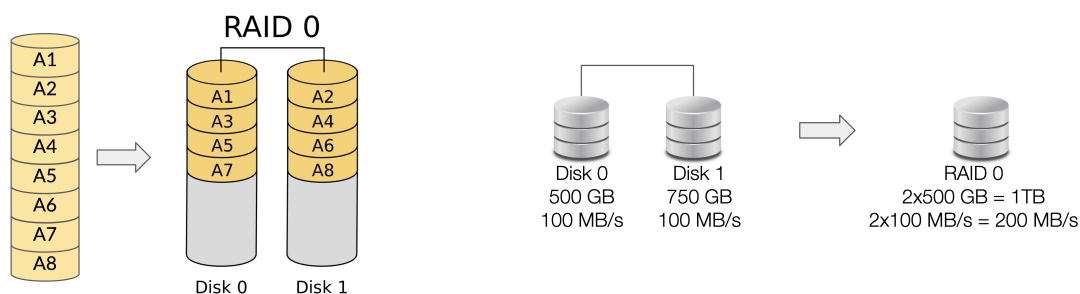
*Strategie adottate:* **Striping** (No mirroring, no checking)

*Numero minimo di dischi richiesti:* 2

*Capacità totale:* la capacità del singolo disco più bassa moltiplicata per il numero di dischi totali

*Throughput:* condizionata dal fatto che la lettura e scrittura avvenga contemporaneamente da tutti i dischi<sup>3</sup>

Privilegia la performance all'affidabilità, dato che perdere un disco implica perdere tutti i dati.



## RAID 1

*Strategie adottate:* **Mirroring** (No striping, no checking)

*Numero minimo di dischi richiesti:* 2

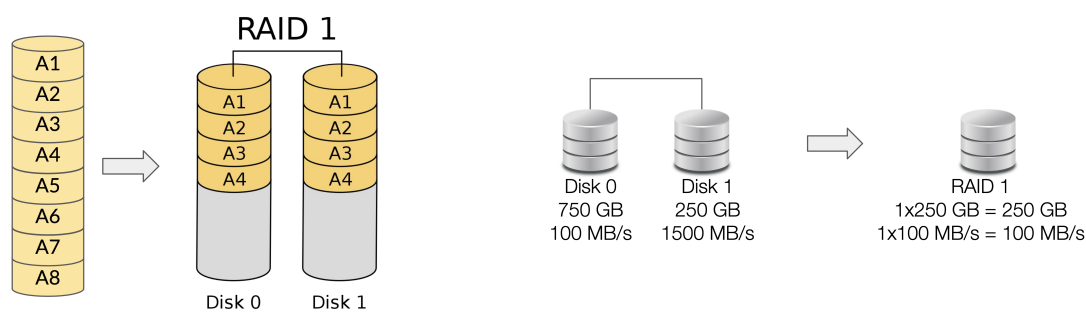
*Capacità totale:* la capacità del singolo disco più bassa

*Throughput:* la throughput del singolo disco più bassa

Privilegia l'affidabilità alla performance, dato che i dati sono al sicuro fintantoché almeno un disco è in funzione.

---

<sup>3</sup>La formula completa è piuttosto articolata e non richiesta dal professore; ho riportato in appendice C una mia proposta a riguardo.



### RAID 1+0

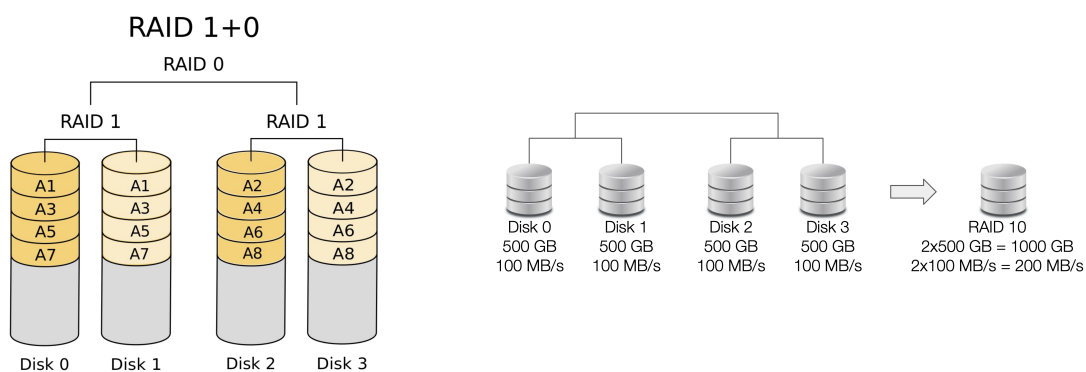
*Strategie adottate:* **Mirroring e striping** (No checking)

*Numero minimo di dischi richiesti:* 4

*Capacità totale:* la capacità dell'unità RAID 1 più bassa moltiplicata per il numero di unità RAID 1 totali

*Throughput:* in lettura la throughput del RAID 0 (veloce), in scrittura la throughput del RAID 1 (lenta)

Mantiene le alte performance del RAID 0 migliorando al contempo l'affidabilità con il mirroring, poiché i dati sono al sicuro fintantoché almeno un disco in ogni unità RAID 1 è in funzione.





## RAID 5

*Strategie adottate:* **Striping e parity checking** (No mirroring)

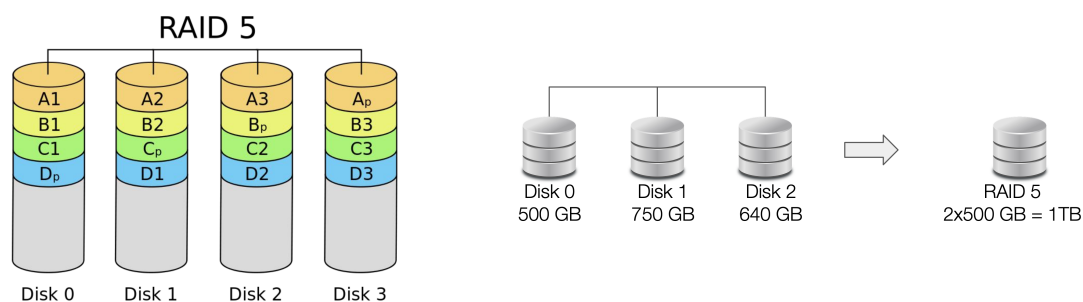
*Numero minimo di dischi richiesti:* 3

*Capacità totale:* la capacità del singolo disco più bassa moltiplicata per il numero di dischi totali meno uno

*Throughput:* non banale a causa della lettura/scrittura dell'informazione sulla parità

Una porzione di ciascun disco viene “sacrificata” in favore dell'informazione sulla parità, dunque i dati possono essere recuperati se un singolo disco si guasta. La parità viene calcolata tramite una catena di XOR:

$$p = D_1 \oplus D_2 \oplus \dots \oplus D_n$$



## RAID 6

*Strategie adottate:* **Striping e parity checking** (No mirroring)

*Numero minimo di dischi richiesti:* 5

*Capacità totale:* la capacità del singolo disco più bassa moltiplicata per il numero di dischi totali meno due

*Throughput:* non banale a causa della lettura/scrittura dell'informazione sulla parità

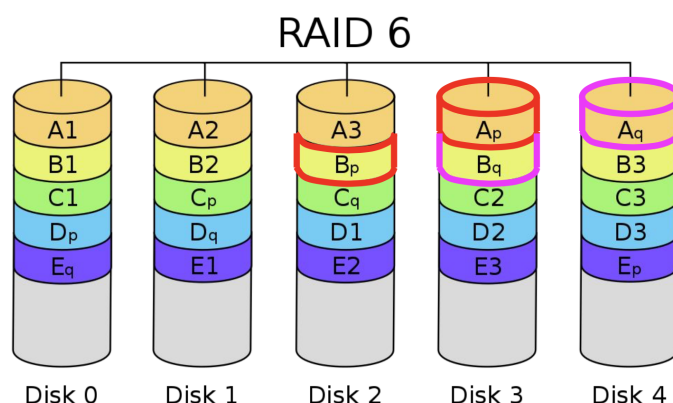
Una porzione di due dischi viene “sacrificata” in favore dell'informazione sulla parità, che sarà codificata in due bit per ogni sezione di dati distribuita su tutti i dischi. Affinché i dati siano recuperabili dopo il guasto di due dischi è necessario che tali bit di informazione sulla parità siano *linearmente indipendenti*, ed esistono dei codici appositi che realizzano ciò, il più famoso dei quali è il codice Reed-Solomon.

Un esempio è il seguente:

$$p = D_1 \oplus D_2 \oplus D_3$$

$$q = D_1 \oplus (D_2 \gg \text{shift}) \oplus (D_3 \gg \text{shift}^2)$$

dove *shift* è un numero intero che, preceduto dal simbolo  $\gg$ , indica lo spostamento lungo il dataset dalla sezione di bit indicata.



### 9.3.2 Crittografia

La **crittografia** è un ambito della matematica che si occupa di codificare le informazioni in modo da renderle sicure e intellegibili solo alle persone autorizzate ad accedervi. Si tratta di una branca molto vasta di cui tratteremo solo i punti salienti che ci interessano nell'ambito del data management.

Essa si fonda sul concetto di *sicurezza*, che è necessaria in tantissimi aspetti: il certificare l'identità di chi sta svolgendo un'azione, lo stabilire a cosa un utente può e non può avere accesso, l'assicurarsi l'integrità dei dati in seguito a un trasferimento e altro ancora. Le principali azioni che coinvolgono il tema della sicurezza sono le seguenti:

- Proteggere i dati salvati su un dispositivo
- Stabilire connessioni sicure con server remoti
- Scambiare informazioni tra due *peer*
- Impedire a utenti malintenzionati di manomettere i dati

Per visualizzare i principali concetti possiamo immaginare, per semplicità, lo scambio di informazione tra due dispositivi, o in altre parole un *messaggio* che viene trasmesso da un utente a un altro utente e viceversa.

Affinché questo scambio sia considerato *sicuro* dovremo porci le seguenti domande:

- Sono sicuro di sapere con chi sto parlando? Posso identificare con certezza la sua identità?
- Qualcun altro può ascoltare il messaggio e intercettare i dati?  
Questa è una delle domande più importanti, dunque rispondiamo subito: *sì, è sempre possibile che qualcuno intercetti l'informazione*, non c'è modo di evitarlo e bisogna scendere a patti con questo fatto, che costituirà la *base* dei sistemi di sicurezza da implementare.
- Il messaggio che ho ricevuto è *identico* a quello inviato? In altre parole: posso assicurarmi dell'integrità del messaggio nel corso del trasferimento?
- L'utente con cui sto dialogando è *responsabile* del messaggio inviato?

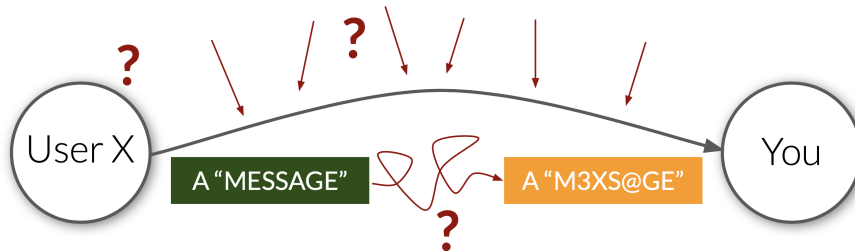


Figura 9.12: Schematizzazione dello scambio del messaggio con tutti i pericoli che ne mettono in pericolo l'integrità e l'affidabilità

Nel contesto di attività e imprese che si occupano di dati su larga scala queste domande sono di fondamentale importanza, ed esse investono ingenti quantità di lavoro e denaro tentando di dare risposte quanto più certe possibili.

Avendo afferrato il motivo per cui la crittografia è così implicata nel data management vediamo di seguito i principali concetti in cui essa si snoda:

1. **Confidenzialità:** assicurarsi che nessuno possa estrarre informazioni pur intercettando l'intera conversazione. Ricordiamo che bisogna sempre dare per certo che qualcuno possa ascoltare i messaggi scambiati.
2. **Integrità:** assicurarsi che il messaggio non sia stato modificato durante il trasferimento.
3. **Autenticità:** verificare che si stia comunicando con l'entità che si pensi stia effettivamente all'altro capo della conversazione.
4. **Identità:** verificare l'identità dello specifico individuo dietro quell'entità.
5. **Non ripudio:** assicurarsi che l'individuo responsabile di una determinata attività non possa negare di essere associata ad essa.

### 9.3.3 Autenticazione e autorizzazione

Gli ultimi tre concetti elencati possono essere inclusi in due aspetti tra loro collegati: quello dell'**autenticazione** e quello dell'**autorizzazione**.

- **Autenticazione:** identificare gli utenti e validare le loro identità. Ciò può essere eseguito in vari modi, il più conosciuto dei quali è l'*autenticazione personalizzata* attraverso username e password, ma ne esistono altri: certificati, chiavi, autenticazioni uniche come l'*SSO* e altre autenticazioni personalizzate come quelle basate sui dati biometrici (impronte digitali, riconoscimenti facciali e altro ancora).
- **Autorizzazione:** concedere a un utente i permessi per accedere a determinate risorse o per utilizzarle.

Di solito l'autenticazione avviene attraverso il collegamento dell'utente a un server, chiamato *authorization server*, che certifica l'ingresso attraverso uno specifico strumento fornito dall'utente; solo dopo che l'autenticazione è avvenuta il sistema verifica i permessi dell'utente attraverso una *Access Control List* in cui sono riportate le risorse a cui esso può accedere, contenute nel *resource server*, e le operazioni che con esse può eseguire. Come detto in precedenza, una volta autenticato e autorizzato a eseguire un'azione l'utente sarà l'unico *responsabile* dell'azione stessa.

Quanto detto sembra semplice e banale, ma nella realtà tutto ciò permea le nostre vite a livelli di cui spesso non ci rendiamo conto: quando paghiamo qualcosa con un sistema come Google Pay o Apple Pay dobbiamo sbloccare il nostro smartphone con qualunque sistema di riconoscimento abbiamo scelto (PIN, impronta digitale, riconoscimento facciale e così via) e nel momento in cui si avvicina il chip NFC al POS viene avviata una comunicazione con la banca sfruttando una serie di layer di sicurezza; solo dopo che tutte le verifiche di autenticazione e autorizzazione vengono eseguite dal sistema la richiesta iniziale, ovvero il pagamento, viene accettata ed eseguita.

### 9.3.4 Codifica e decodifica dell'informazione

Come abbiamo detto, tutti i messaggi scambiati attraverso strumenti informatici sono costantemente esposti. Scesi a patti con ciò, capiamo che l'unico modo per evitare che l'informazione sia estratta dal messaggio è modificare quest'ultimo in modo che chi lo ascolti non sia comunque in grado di capirlo.

L'idea più basilare è quella di applicare una *maschera* al messaggio, detta **chiave**, che ha il ruolo di *mescolarlo* e *riassemblarlo*: in crittografia queste azioni prendono il nome di **cifrare** e **decifrare**, o anche di **codificare** e **decodificare**.

Se il messaggio è espresso da un insieme di bit, la chiave sarà a sua volta un insieme di bit della stessa lunghezza. Un esempio molto semplice di codifica è dato dall'operazione XOR applicata al messaggio; la comunicazione avverrebbe nel seguente modo: prima che il messaggio sia inviato dall'utente *A* all'utente *B*

esso viene codificato dalla chiave eseguendo un'operazione bitwise XOR; l'utente *B*, una volta ricevuto il messaggio, decodifica il messaggio rieseguendo la stessa operazione con la stessa chiave che è comune ai due utenti.

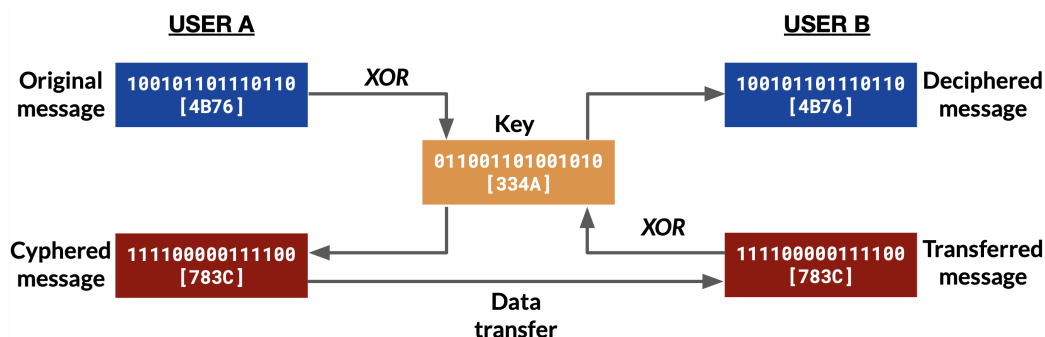
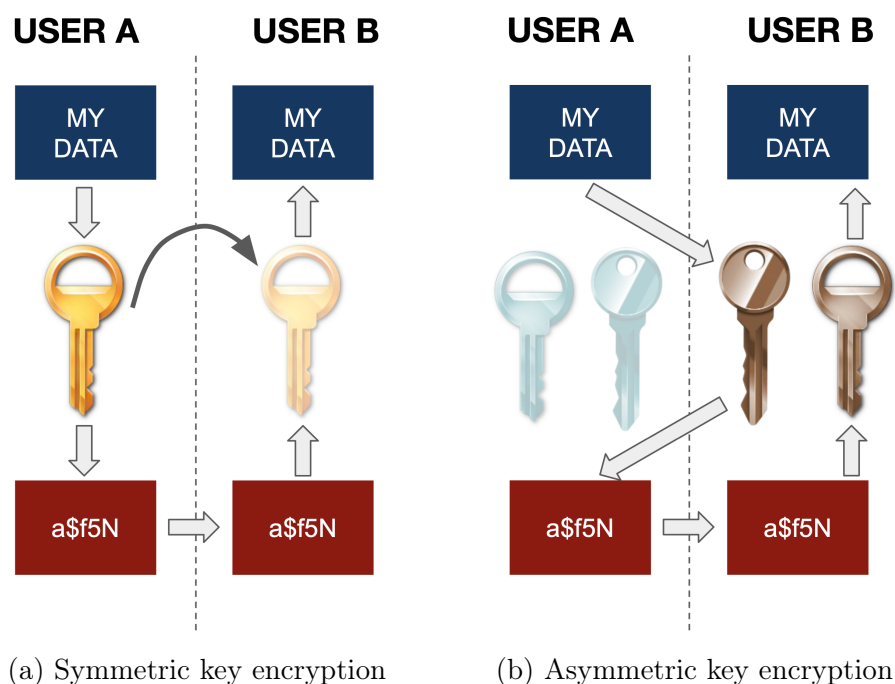


Figura 9.13: Schema di codifica e decodifica con chiave in comune

La scelta dello XOR è un'ovvia semplificazione, ma nella realtà questa tecnica è realmente adottata usando degli algoritmi specifici come l'AES-128, l'AES-192 e l'AES-256. Si tratta di un metodo semplice da implementare ma che è al contempo soggetto a più rischi a causa del fatto che i due utenti devono condividere la stessa chiave: quest'ultima non può ovviamente essere comunicata tra i due, dato che altrimenti si tornerebbe al problema di partenza; l'unico modo realmente efficace per condividere la chiave è trasferirla a mano, che è un metodo ovviamente scomodo.

Per tale ragione questo tipo di crittografia, chiamata **symmetric key encryption** o anche **private-key cryptography**, non è attualmente applicata nei sistemi di sicurezza a più alto livello, se non come secondo layer aggiuntivo. La più utilizzata oggi è sicuramente la **asymmetric key encryption**, detta anche **public-key cryptography**, che fa uso di due chiavi: una *pubblica* e una *privata*. Esse sono prodotte *insieme* dallo stesso algoritmo (*RSA*, *ECC* e altri ancora) in modo che da una chiave sia impossibile risalire all'altra e che entrambe possano agire su un messaggio in un'unica "direzione": la chiave pubblica ha l'unico scopo di *cifrare* i dati, mentre quella privata può solo *decifrare* gli stessi.

Secondo questo protocollo l'utente rende disponibile la sua chiave pubblica in modo chiaro, e chiunque intenda inviargli un messaggio codificherà quest'ultimo attraverso tale chiave; il punto cruciale è che il messaggio cifrato non potrà essere decifrato attraverso la stessa chiave, ma solo attraverso la chiave privata che è però posseduta esclusivamente dall'utente destinatario del messaggio. Si tratta di un sistema di codifica/decodifica molto più lento e complesso in termini computazionali, ma certamente più sicuro, al punto da essere considerato uno standard al giorno d'oggi.



### 9.3.5 Funzioni hash

Finora abbiamo discusso dei concetti di *autenticità*, *identità* e *non ripudio* secondo i passaggi che definiscono l'autenticazione e l'autorizzazione, così come il concetto di *confidenzialità* attraverso la codifica e decodifica di messaggi possibili solo per gli utenti finali. Resta ancora da affrontare la tematica dell'*integrità* dell'informazione scambiata.

Di questo si occupano le **funzioni hash**, applicando il cosiddetto *hashing* ai dati, ovvero un loro rimescolamento al punto che la forma originale *non possa più essere riprodotta*. Tutti gli algoritmi dedicati all'hashing (alcuni esempi sono l'*MD5*, lo *SHA-1\** e lo *SHA-2\**) producono alla fine un insieme di dati, chiamato **digest**, dalla lunghezza fissata e uguale per tutti. Modificare anche solo di poco l'informazione in input cambierà completamente il digest finale, com'è possibile osservare in figura 9.15.

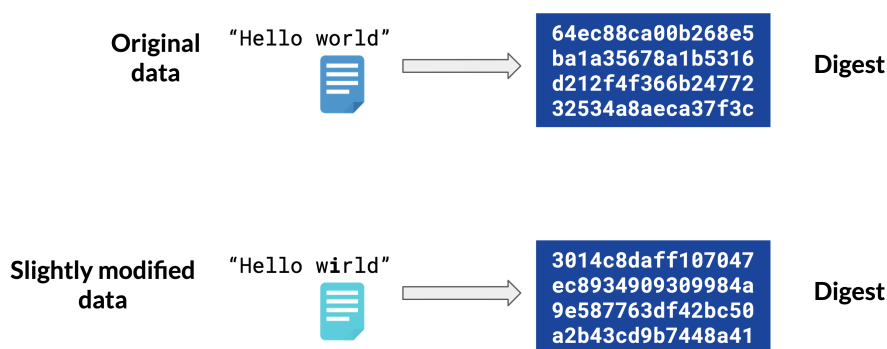


Figura 9.15: Digest di due insiemi di dati molto simili

Ciò che gli algoritmi devono in ogni modo impedire, e quindi ciò su cui si concentrano maggiormente i creatori di tali algoritmi, sono le **hash collision**, ovvero quei casi in cui due insiemi di dati differenti danno luogo allo stesso digest. L'importanza delle funzioni di hash, infatti, è quella di produrre dei codici *univoci*, e ogni conflitto annullerebbe il loro scopo primario.

Vediamo ora i tre principali motivi per cui è usato l'hashing:

- **Protezione dei dati:** utile per il salvataggio di dati sensibili, come username e password, all'interno dei sistemi informatici: questi dati, infatti, non possono essere riportati in modo chiaro per questioni di sicurezza, quindi subiscono l'hashing e il digest risultante è ciò che viene effettivamente salvato; nei processi di autenticazione, infatti, ciò che viene confrontato dal sistema sono gli hash, quelli in input e quelli salvati in memoria.
- **Verifica dell'integrità dei dati:** per assicurarsi che i dati ricevuti non siano stati in qualche modo manomessi è utile eseguire un check attraverso l'hash associato a tale pezzo di informazione. Ad esempio, scaricando un file da internet, potrebbe essere richiesto all'utente di confrontare l'hash del file con quello previsto per accertarsi che i due coincidano, e sia quindi possibile aprire il file in sicurezza con la consapevolezza dell'integrità di quanto sia all'interno.
- **Firme digitali:** a causa della loro unicità e irriproducibilità gli hash sono utilizzati per certificare documenti archiviati in sistemi digitali attraverso firme digitali.

## 9.4 File System

Esistono tre principali soluzioni per memorizzare e lavorare con dataset:

1. **File:** è il più conosciuto e utilizzato. I *file* sono insiemi di blocchi di dati gestiti da un sistema chiamato *File System*, che distribuito su più unità di memoria (scalabilità orizzontale) agirà come un *Distributed File System*.
2. **DataBase:** ogni singolo elemento è accessibile a un livello *granulare* e modificabile attraverso dei sistemi chiamati *DataBase Management System*; se distribuiti su più unità di memoria i DataBase sono chiamati *Distributed DataBases*.
3. **ObjectStore:** i dati sono salvati in qualunque forma arbitraria (file, blocchi di lunghezza specifica, interi sistemi operativi) all'interno di contenitori chiamati *bucket*; sono utilizzati principalmente nei *cloud storage*.

In questa sezione tratteremo dei file system, mentre nel prossimo parleremo di database e database management system.

### 9.4.1 Caratteristiche

Il ruolo di un file system è quello di gestire e tenere traccia della posizione dei dati salvati in memoria in forma di file attraverso una struttura di tipo **gerarchico**, facendo uso di cartelle (chiamate anche *directory*) e sottocartelle. Le sue principali caratteristiche si possono riassumere in tre aspetti:

1. **Salvataggio dei dati:** i dati sono fisicamente memorizzati in forma di *blocchi* di lunghezza fissata, i quali vengono distribuiti all'interno dei dispositivi di memoria. Il file system ha il compito sia di suddividere e allocare in questo modo i file in memoria, sia quello di “tenere in mente” tutte le posizioni dei blocchi una volta che l'utente intende accedere al file in questione.
2. **Astrazione:** il file system è in grado di identificare i dati attraverso determinate caratteristiche quali il *nome* del file, il *percorso* organizzato secondo struttura gerarchica e i *metadati* attribuiti al file.
3. **Operazioni di basso livello:** il file system ha il compito di interfacciarsi con il sistema operativo per permettere le operazioni di *basso livello* sui blocchi di archiviazione, come l'apertura, la chiusura, la lettura, la scrittura, la ricerca e altro ancora. Tale interfaccia segue degli standard precisi, il più diffuso dei quali è il **POSIX** (*Portable Operating System Interface for Unix*).

### 9.4.2 Allocazione dello spazio

La principale distinzione tra i vari file system riguarda il modo in cui viene allocato lo spazio per salvare in memoria i file. Vedremo di seguito tre tipi di allocazione procedendo per scala di efficienza.

#### Allocazione contigua

I vari file sono salvati in modo *contiguo*, nel senso che i vari blocchi che costituiscono i singoli file sono disposti fisicamente uno accanto all'altro. L'accesso ai file, di conseguenza, sarà di tipo **sequenziale**.

Il limite di questo tipo di allocazione è evidente: nel momento in cui si vuole ampliare il contenuto di un file bisogna tenere conto che, oltre un certo punto, verrà incontrato un blocco già occupato che costituisce l'inizio di un altro file, e non può quindi essere sovrascritto. Per questa ragione un file system basato su tale allocazione si adatta ai nastri magnetici, in cui di norma i file vengono salvati per essere conservati sul lungo periodo, senza mai essere modificati.



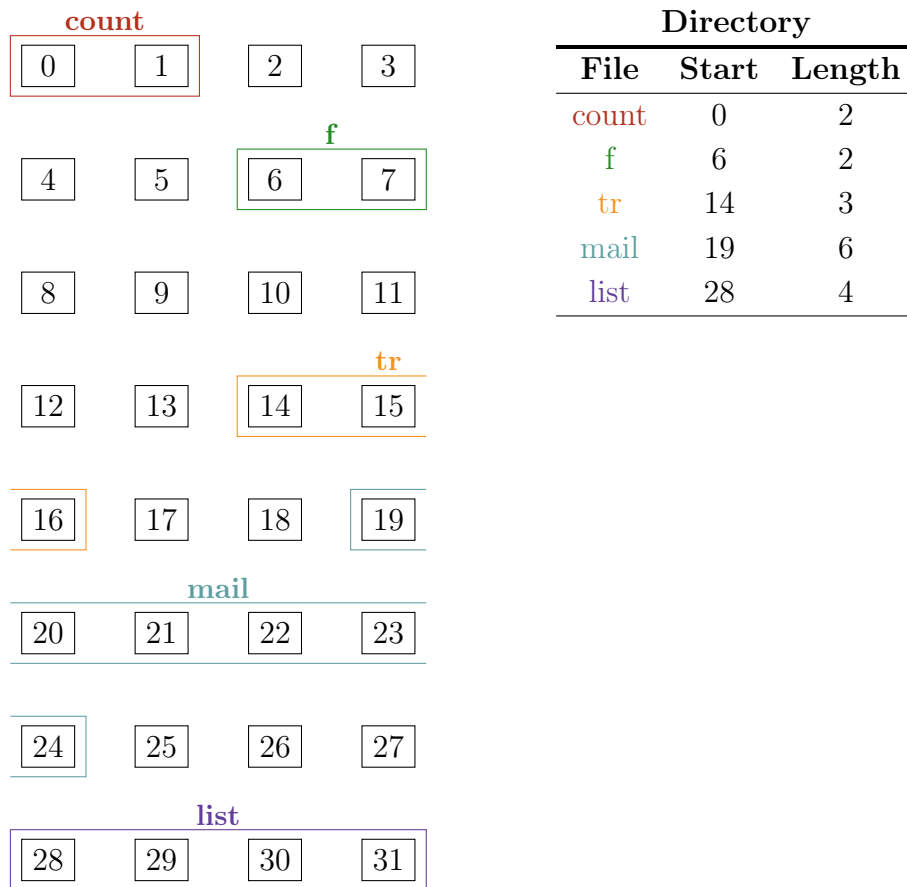


Figura 9.16: Esempio di allocazione contigua con cinque file

### Allocazione collegata

Attraverso questo step in avanti è possibile “saltare” da un blocco all’altro per memorizzare un unico file. Questo è possibile inserendo in ogni blocco l’informazione su dove si trovi il blocco di dati successivo, dunque una piccola parte della memoria deve essere “sacrificata” per permettere al file system di muoversi e recuperare le varie informazioni.

Rispetto all’allocazione contigua qui è possibile una migliore ottimizzazione dello spazio a disposizione, non essendo vincolati a una serie di blocchi contigui per comporre il file.

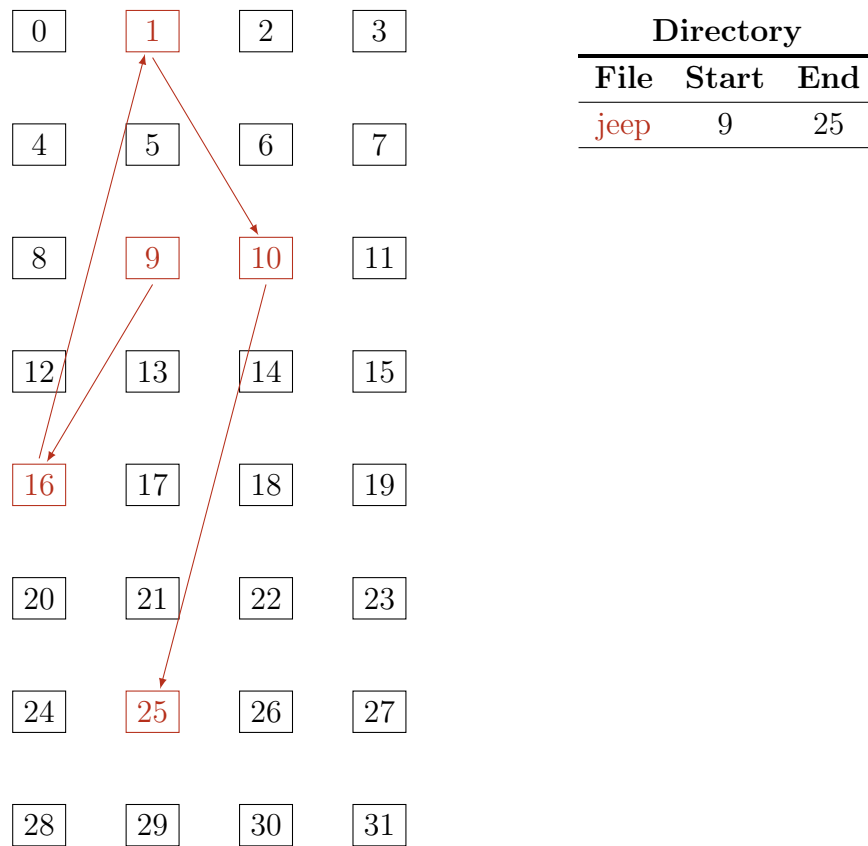


Figura 9.17: Esempio di allocazione collegata

### Allocazione indicizzata

L'allocazione collegata ottimizza certamente lo spazio nel disco ma non il tempo nelle operazioni che coinvolgono i file, come lettura e scrittura: ogni volta che un'operazione viene richiesta dall'utente, infatti, è necessario che il sistema parta dal blocco iniziale e prosegua “rimbalzando” tra tutti i blocchi successivi del file fino a raggiungere la destinazione voluta. L'ottimizzazione del tempo può essere ottenuta mediante un nuovo tipo di allocazione, detta *indicizzata* in quanto fa uso dei blocchi diretti e indiretti, con gli ultimi aventi la funzione di indicare le posizioni dei dati.

Il primo blocco è costituito come segue: la parte iniziale è formata dai *meta-dati* relativi al file; seguono poi i *blocchi diretti*, ovvero quell'insieme di dati che fornisce gli indirizzi in cui si trovano i dati, ovvero i *puntatori*; eventualmente possono esserci anche i *blocchi indiretti singoli*, ovvero blocchi che puntano a loro volta ad altri blocchi che puntano ai dati, e con la stessa logica vengono realizzati anche i *blocchi indiretti doppi* e così via.

Facciamo l'esempio di un file system molto simile al vecchio Berkley Fast File System: a ogni file sono dedicati dodici blocchi diretti e un blocco indiretto, tutti formati da 4 *kB*. La domanda che ci poniamo è: quanto grande può essere un file

gestito da questo sistema, considerando che l'indice di un indirizzo di memoria occupa 4 byte?

I dodici blocchi diretti contengono dati, dunque offriranno un totale di

$$12 \times 4 \text{ kB} = 48 \text{ kB}$$

Resta ancora il blocco indiretto, che contiene solo indirizzi di memoria. Se il blocco è formato da 4 kB e ogni indirizzo sono 4 B il numero totale di indirizzi contenuti nel blocco è

$$\frac{4 \text{ kB}}{4 \text{ B}} = 1000$$

Saranno così puntati 1000 blocchi diversi, la cui memoria totale è

$$1000 \cdot 4 \text{ kB} = 4 \text{ MB}$$

che, sommata alla precedente, darà un totale pari a

$$4 \text{ MB} = 48 \text{ kB} = 4048 \text{ kB}$$

Ne segue che, per questo file system, ogni file può avere come dimensione massima poco più di 4 MB.

Immaginiamo ora che sia disponibile un ulteriore blocco e che sia indiretto doppio. Questo significa che esso contiene gli indirizzi di mille blocchi che a loro volta conterranno gli indirizzi di mille blocchi di dati, per un totale di

$$1000 \cdot 1000 \cdot 4 \text{ kB} = 4 \text{ GB}$$

È evidente, dunque, che aumentare il layer di blocchi indiretti aumenta esponenzialmente la memoria dedicata a un file.

### 9.4.3 Limiti

Dai ragionamenti appena fatti capiamo che un file system è limitato principalmente dallo spazio che può dedicare a un singolo file. Un ulteriore limite di memoria è dato dallo spazio totale che un file system può gestire per una singola unità di memoria, dovuto al fatto che per “contare” i blocchi di file all'interno fa uso di un contatore con un numero limitato di bit.

Altre differenze che distinguono i file system sono i metadati che vengono salvati per ciascun file. Di seguito vediamo una tabella che confronta alcuni tra i file system più famosi relativamente alle caratteristiche appena citate.

|                       | <b>FAT32</b> | <b>NTFS</b> | <b>exFAT</b> | <b>HFS</b> | <b>HFS+</b> | <b>ext2</b> | <b>ext4</b> |
|-----------------------|--------------|-------------|--------------|------------|-------------|-------------|-------------|
| <b>File</b>           | 4 GB         | 16 EB       | 16 EB        | 2 GB       | 8 EB        | 16 GB/2 TB  | 16 GB/16 TB |
| <b>Volume</b>         | 16 TB        | 16 EB       | 64 ZB        | 2 TB       | 8 EB        | 32 TB       | 1 EB        |
| <b>Nome file</b>      | 255 UCS-2    | 255         | 255 UTF-16   | 31 B       | 255 UTF-16  | 255 B       | 255 B       |
| <b>Proprietario</b>   | No           | Sì          | No           | No         | Sì          | Sì          | Sì          |
| <b>Permessi</b>       | No           | Sì          | No           | No         | Sì          | Sì          | Sì          |
| <b>Ultimo accesso</b> | Sì           | Sì          | Sì           | No         | Sì          | Sì          | Sì          |

Un altro limite da considerare è il fatto che il file system non può dare informazioni sul contenuto all'interno del singolo file, e sta quindi all'utente sfruttare l'organizzazione gerarchica ad albero fornita dal sistema per ordinare i dati nel modo più conveniente. Ciò non consente di eseguire determinate operazioni, come l'applicazione di filtri basati sul contenuto dei file.

Eseguiamo un bilancio finale in base a quanto abbiamo detto:

| Pro                                                                                              | Contro                                                                                   |
|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Può gestire una grande quantità di dati                                                          | Scalare una crescente quantità di dati secondo la visione gerarchica è complesso         |
| Le operazioni di basso livello tramite POSIX sono molto veloci                                   | Nessuna struttura sottostante quella dei file è accessibile                              |
| Offre una visione gerarchica di tutti i dati                                                     | Non è possibile raggruppare, selezionare e filtrare i dati in base al contenuto dei file |
| Le singole entità (i file) possono essere facilmente trasferite, copiate, condivise e processate |                                                                                          |

## 9.5 File system distribuito

Un punto importante che abbiamo in parte già trattato è il modo in cui un file system è capace di rappresentare i dati acquisiti all'interno dei dischi di memoria. La questione che ci poniamo ora è la *scalabilità* del sistema rispetto a un numero sempre crescente di byte di memoria.

Sappiamo che per un sistema informatico la scalabilità può essere verticale o orizzontale. Per quanto riguarda la prima, in principio, non vi è alcun problema: il file system si interfaccia direttamente con la macchina, e sarà quindi in grado di “vedere” nuova memoria disponibile sulla stessa per allocare i file. Quel di cui dobbiamo occuparci, invece, è la scalabilità orizzontale, perché in quel caso parliamo di macchine differenti e autonome.

Per applicare un file system a un sistema scalato in modo orizzontale dobbiamo spostarci dai file system visti finora, chiamati *file system locali*, ai **file system distribuiti**, indicati con *DFS* (**D**istributed **F**ile **S**ystems). In termini tecnici indichiamo come *cluster* un insieme di risorse di calcolo, ovvero di computer chiamati in questo contesto *nodi* del cluster; tipicamente l'utente che ha accesso a questo sistema “vive” al di fuori di esso, nel senso che è in grado di connettersi attraverso un proprio computer avendo scaricato un programma dedicato a questa funzione, chiamato *client*. I singoli nodi sono collegati tra loro attraverso una rete (progettata per essere *molto* veloce) chiamata *communication network*, che è anche quella a cui accede direttamente l'utente esterno: questo significa che l'utente non ha una visione *atomica* dei singoli nodi nel cluster.

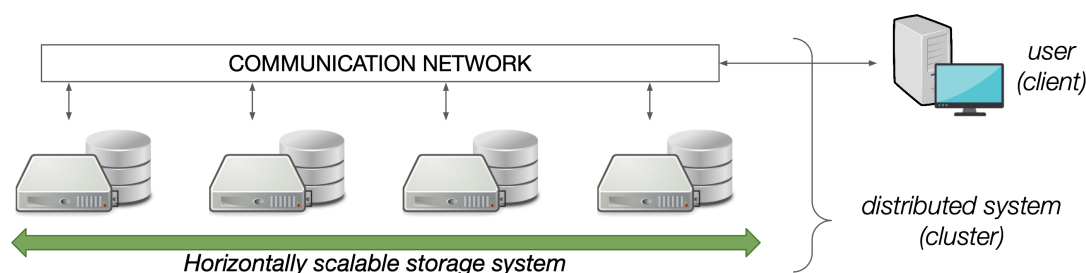


Figura 9.18: Schema di un sistema di archiviazione scalato orizzontalmente

Per comprendere il funzionamento dei DFS dovremo astrarre il concetto di file system visto finora: non si tratta di sistemi associati ai singoli dispositivi in sé, ma condivisi nel cluster in modo che le proprietà e le azioni che conosciamo siano mantenute, come l'accesso a un file per la lettura e scrittura e così via.

### 9.5.1 Funzionamento

Esistono diversi DFS con caratteristiche diverse e, di conseguenza, adatti in contesti differenti; alcuni esempi sono i seguenti:

- **NFS** (*Network File System*): costruito per sistemi operativi UNIX, è il più diffuso e appropriato a livello utente
- **AFS** (*Andrew File System*): adatto per scopi accademici in ambienti di ricerca
- **Lustre**: realizzato appositamente per l'ambito HPC (*High Performance Computing*) in modo da massimizzare il throughput e gestire un'ingente mole di dati
- **EOS** (*Elastic Object Storage*): sviluppato al CERN per gestire principalmente una grande quantità di dati non strutturati

Tutti i DFS, però, hanno lo stesso funzionamento di base, che conosciamo già dai file system locali: i dati (insieme ai relativi metadati) sono suddivisi in file, a loro volta spezzati in blocchi di memoria dalla dimensione fissa distribuiti in posizioni molto diverse ma sempre rintracciabili tramite indici e puntatori.

Le differenze principali rispetto ai file system locali sono tre:

1. I block size vengono suddivisi su macchine diverse, dunque per la posizione non deve essere specificato solo l'indirizzo di memoria ma anche la macchina stessa in cui si trova il blocco
2. Dato che le operazioni devono agire su macchine diverse per lavorare sullo stesso file la latenza aumenta inevitabilmente, perché oltre a quella insita nelle componenti meccaniche dei computer si aggiunge anche quella relativa al communication network, che è di molto superiore

3. A causa del precedente punto i block size sono di dimensioni maggiori rispetto a quelli tipici dei file system locali, ovvero dell'ordine dei megabyte, in modo da massimizzare la quantità di dati da “estrarre” da ogni posizione<sup>4</sup>

Abbiamo descritto nella pratica come opera un file system distribuito per l'allocazione e il recupero dei dati in memoria. Guardiamo ora, infine, tre requisiti *ideali* a cui bisognerebbe tendere nella realizzazione di un suddetto sistema:

1. **Trasparenza**, ovvero la semplicità di utilizzo per l'utente che non dovrebbe provare alcuna differenza nel lavorare con un DFS rispetto a un file system locale; detto in altre parole, la dispersione dei dati su più computer deve essere il meno evidente possibile.
2. **Tolleranza ai guasti**, ovvero la capacità di mantenere attive tutte le operazioni nonostante il cedimento di alcune infrastrutture.
3. **Scalabilità**, ovvero l'estensibilità a nuovi nodi nel cluster in modo semplice e pratico.

### 9.5.2 Esempio: HDFS

L'**HDFS** (*Hadoop Distributed File System*) è un DFS open-source realizzato nel 2006 nel tentativo di emulare tramite reverse engineering il file system introdotto da Google tre anni prima, chiamato *GFS* (*Google File System*). Nasce con lo scopo di gestire ed elaborare grandi quantità di dati su *commodity hardware*, cioè dispositivi relativamente economici e facilmente reperibili sul mercato; anche per questa ragione la comunicazione è basata su protocolli standard e largamente diffusi, chiamati TCP/IP.

In questo sistema è presente un nodo speciale chiamato **namenode**, che contiene i metadati di tutti i file ed ha al contempo il ruolo di “prendere le decisioni”, ovvero di accettare o rifiutare le richieste di operazioni che vengono inviate dagli utenti; tutti gli altri nodi, detti **datanode**, ospitano i blocchi di file grandi ciascuno 128 MB.

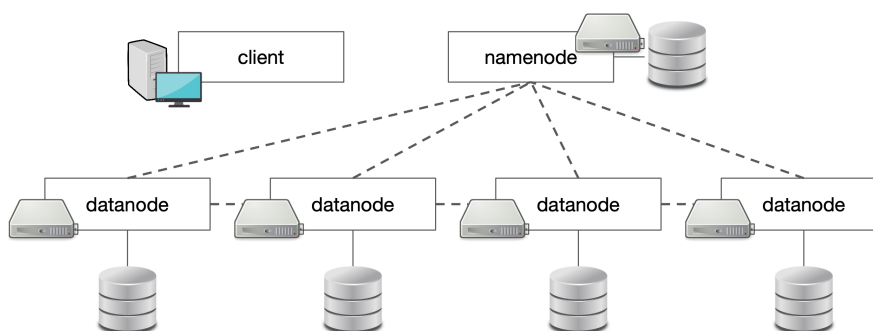


Figura 9.19: Schema del sistema HDFS

<sup>4</sup>Questa caratteristica è anche giustificata dal fatto che i distributed file system agiscono su sistemi realizzati per memorizzare un'ingente quantità di dati, quindi proporzionalmente dovranno crescere anche le dimensioni dei block size rispetto ai local file system.

Ogni operazione può essere articolata in quattro passaggi:

1. Il client invia la propria richiesta (che sia questa lettura, scrittura, cancellazione e così via) direttamente al namenode
2. Il namenode verifica le autorizzazioni e i permessi, e solo dopo aver concluso questo passaggio fornisce al client le posizioni in cui sono situati i data block contenenti i dati interessati dall'operazione richiesta dall'utente
3. Il client agisce direttamente sui datanode contenenti tali blocchi senza passare dal namenode
4. I datanode segnalano direttamente al client che l'operazione è conclusa

Dato questo sistema, osserviamo come esso rispetti i tre requisiti ideali scritti in precedenza: la trasparenza è data dal fatto che l'utente non deve interessarsi di tutti gli step intermedi, ma ha la facoltà di compiere un'azione e attendere la risposta di completamento una volta che questa viene eseguita dal sistema; la scalabilità, invece, è dovuta al design stesso dell'HDFS, che rende particolarmente semplice il collegamento di una nuova macchina agli altri datanode e al namenode.

Per quanto riguarda la tolleranza ai guasti è necessario suddividere il problema in tre parti, sottolineando tre diversi tipi di guasti che possono affliggere il sistema:

- **Instabilità del network:** alcuni collegamenti del sistema saltano, isolando uno o più datanode dal resto del network e causando così la formazione di una *network partition*.

Ciascun datanode invia ripetutamente<sup>5</sup> un segnale al namenode, come una sorta di “battito” che indichi la sua presenza nel network e la disponibilità all'accesso dei dati. Nel momento in cui questo battito viene a mancare il namenode esclude la possibilità di operare su quel nodo fino a quando il battito non viene ripristinato, confermando la risoluzione del guasto alle linee di comunicazione.

- **Guasto di un datanode:** un datanode si guasta, perdendo così anche i dati al suo interno.

I data block sono copiati su nodi diversi in modo che anche se un datanode si guasta, i dati non vengono comunque persi. Non si tratta di un RAID vero e proprio, perché è il sistema stesso a essere costituito da più macchine, quindi vengono utilizzate le stesse per copiare i blocchi senza l'utilizzo di ulteriori dischi di memoria esterni. Di default vengono eseguite tre copie di un blocco, dunque uno stesso blocco si trova in tre nodi diversi, ma è possibile aumentare questo numero a discrezione dell'utente.

---

<sup>5</sup>Tipicamente nell'ordine dei decimi di secondo.

- **Guasto del namenode:** il namenode si guasta, perdendo così accesso ai metadati e all'interfaccia col client.

Si tratta senza dubbio del peggior guasto possibile a causa del design stesso sistema, trattandosi di uno *SPOF* (*Single Point Of Failure*). Con la perdita del namenode, infatti, non vengono persi i dati in sé, ma viene persa l'informazione sul come comporre i file a partire dai blocchi. Affinché l'intero dataset non venga perso viene sempre eseguita una copia di backup del namenode in modo che questo possa essere sostituito; in questo caso verrà inevitabilmente meno il principio della trasparenza, perché il sistema sarà costretto ad arrestarsi e sarà necessario l'intervento fisico di qualcuno affinché scolleghi il server guasto, crei un'ulteriore copia di backup del namenode (perché è sempre necessario avere una copia) e colleghi il nuovo namenode.

### 9.5.3 Teorema CAP

Per i sistemi distribuiti<sup>6</sup> (non solo quelli di archiviazione visti finora, ma anche quelli di elaborazione) è valido un teorema, dimostrato matematicamente da Eric Brewer e per questo motivo chiamato *teorema di Brewer*, anche se è meglio conosciuto col nome di **teorema CAP** dall'acronimo dei tre concetti principali espressi dal teorema:

1. **Consistency:** *ogni* richiesta da parte di *ogni* client a *qualsiasi* datanode deve risultare nella **stessa** copia dell'informazione salvata.

Come sappiamo, infatti, gli stessi blocchi di memoria vengono copiati e distribuiti su nodi diversi, dunque se due client accedono contemporaneamente e vogliono agire sullo stesso file potranno comunque accedere a due datanode diversi, a patto che, ovviamente, essi restituiscano la stessa identica informazione, da cui il principio appena riportato di *consistency*. Per applicare ciò viene introdotta una "linea d'attesa" in cui porre l'utente subito dopo che modifica un file, ad esempio attraverso la scrittura: nel momento in cui i data block su cui ha agito il client vengono modificati, prima che egli possa eseguire qualsiasi altra azione il sistema viene notificato della modifica e va ad aggiornare tutte le altre copie alla stessa versione, e solo dopo che questo processo è concluso sarà possibile per il client proseguire con un'altra operazione.

---

<sup>6</sup>Essi sono definibili come quei sistemi composti da più entità ma che sono considerati come un'unica entità dal punto di vista di un utente esterno al sistema stesso.



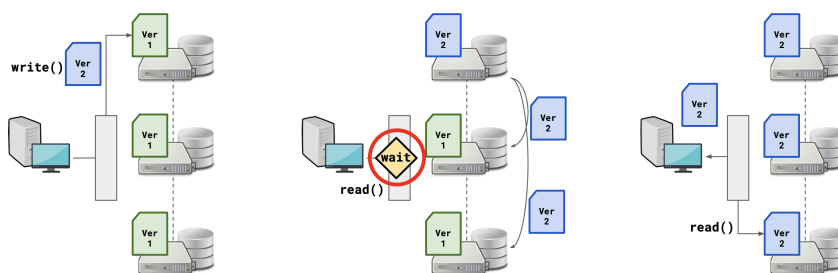


Figura 9.20: Esempio di consistency in caso di scrittura su un file

2. **Availability:** a *ogni* richiesta da parte di *ogni* client deve corrispondere una risposta *immediata, indipendentemente* dallo stato dei dati e dal non funzionamento di alcuni dei nodi del network.

Secondo questo principio il sistema deve essere in grado sia di sostenere le richieste di più client allo stesso tempo, sia di minimizzare il più possibile la latenza, tentando di annullare ogni forma di attesa nella risposta. Per la prima caratteristica è necessario che sia garantita una sufficiente dispersione dei data block nei vari nodi, in modo che più utenti possano accedere allo stesso file, persino se qualche nodo è irreperibile, che sia per un guasto alla linea o al datanode stesso; per la seconda caratteristica, invece, è necessario che sia eliminata ogni forma di attesa nella comunicazione tra network e client.

Per stressare su quest'ultimo aspetto guardiamo all'esempio proposto in figura 9.21: il client scrive su un file, modificandolo in una nuova versione; immediatamente dopo esegue la lettura del medesimo file: per il principio dell'availability ogni nodo contenente l'informazione richiesta invierà la risposta il prima possibile, ma questa potrebbe essere data sia dal nodo in cui è appena stato sovrascritta la nuova versione sia da un altro nodo in cui la versione non è stata aggiornata; non sarà pertanto possibile stabilire quale versione giungerà in lettura al client, trattandosi di una race competition.

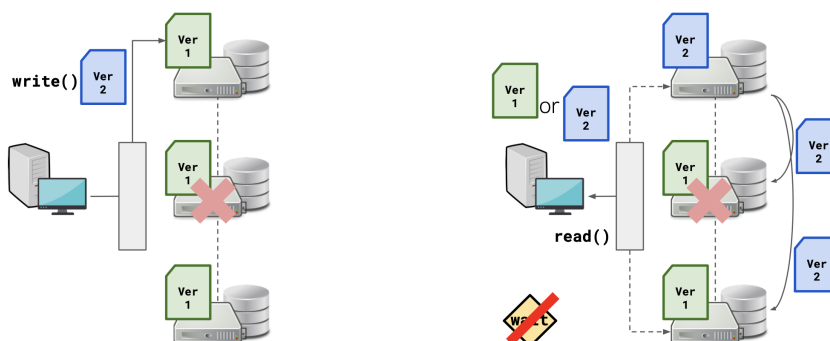


Figura 9.21: Esempio di availability in caso di scrittura su un file e successiva lettura

3. **Partition tolerance:** il sistema *deve* continuare a operare *indipendentemente* dai tagli alle linee di comunicazione tra i nodi, ovvero alla formazione di *network partition*.

Riprendendo gli esempi visti prima, nel momento in cui vogliamo scrivere su un file aggiornandolo a una versione nuova è necessario che almeno un datanode contenente il blocco interessato sia sempre disponibile al client. Come vedremo di seguito, se questa condizione è soddisfatta dovremo scegliere tra due possibilità: o preferiamo l'attesa della risposta alla lettura, in modo da avere una visione coerente in tutto il sistema della stessa versione del file aggiornato, oppure scegliamo la risposta più veloce possibile accettando una race competition.

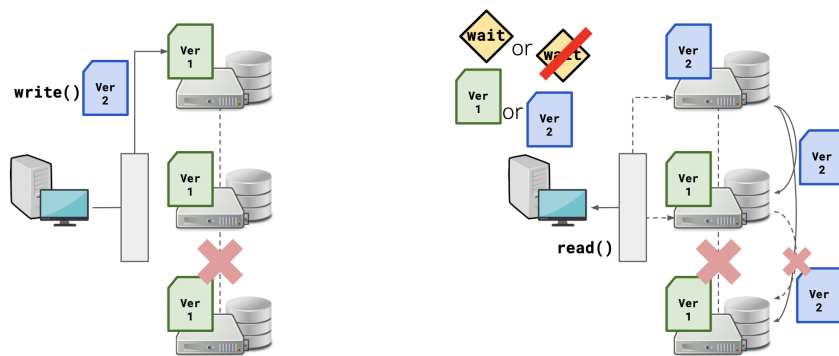


Figura 9.22: Esempio di partition tolerance in caso di scrittura su un file

Il teorema afferma quanto segue:

#### Teorema CAP

È **impossibile** che in un sistema distribuito siano fornite *contemporaneamente* più di **due** delle tre garanzie CAP (*consistency*, *availability* e *partition tolerance*)

In un sistema distribuito non è possibile evitare la partition tolerance, essendo i sistemi realizzati proprio in modo che il cluster sia (per quanto possibile) sempre operativo, anche in caso di problemi alle linee di comunicazione; ciò riduce la scelta a due possibilità:

- **Sistema CP:** è tendenzialmente più lento in fase di risposta ma garantisce coerenza nella lettura da parte di più client
- **Sistema AP:** è veloce nella lettura ma non vi è alcuna garanzia nel fatto che i dati disponibili siano aggiornati all'ultima versione

La scelta tra queste due versioni dipende da ciò che si preferisce privilegiare: nel caso di sistemi ad alto livello di computazione, come il *Lustre*, si preferisce una lettura rapida di dati che vengono costantemente aggiornati, quindi il sistema adottato è quello AP; per sistemi che richiedono invece un maggiore livello di sicurezza complessiva, come ad esempio sistemi bancari, è preferibile un CP in cui si è sempre aggiornati sullo stato dei dati e non vi sono disallineamenti nella lettura tra un dispositivo e un altro.

Come detto, la scelta AC non è una strada percorribile per i sistemi distribuiti, ma è comunque utilizzato in caso di sistemi scalabili verticalmente, dove non è necessario aggiungere nuovi nodi che concorrono alla distribuzione del sistema, ma piuttosto nuovi elementi di memoria sempre più performanti che possano garantire entrambe le condizioni di availability e consistency.

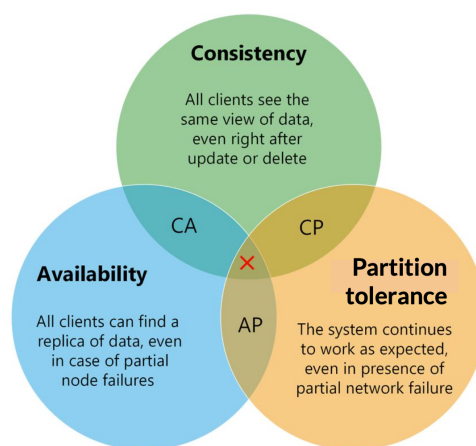


Figura 9.23: Schema del teorema CAP

## 9.6 Database

Una generica definizione di **database** è la seguente:

Un **database** è un insieme di *record* (generalmente correlati tra loro) organizzati e processati da un *sistema di gestione* dedicato

Solitamente nel termine *database* includiamo tre concetti:

1. I *record*
2. Le *collezioni* che raggruppano i singoli record
3. Le *relazioni* che sussistono sia tra record della stessa collezione sia tra record di collezioni diverse

Il sistema di gestione dedicato è chiamato **database management system**, indicato con *DBMS*, le cui principali funzionalità sono le seguenti:

1. *Creare e definire* dati
2. *Selezionare ed effettuare query* sui dati
3. *Manipolare e processare* dati

È importante evidenziare quali siano le differenze principali nella gestione tramite file system e tramite DBMS:

1. La scala a cui agiscono i due software è diversa: se con i file system l'utente può accedere unicamente ai file, ovvero all'intero insieme di record, con i DBMS le operazioni possono avvenire a livello dei *singoli record*
2. L'idea stessa di gestione dati cambia: i dati non sono più "sparpagliati" tra file e directory diverse, ma situati tutti all'interno di un'unica entità (il database, appunto), in cui sono incluse anche le relazioni tra dati stessi
3. Nell'analisi dati i differenti algoritmi applicati allo stesso set di dati non richiedono una nuova copia del file con i suddetti, ma sarà possibile selezionare un sottoinsieme di dati dal dataset senza necessità di replicarli e occupare nuova memoria

### 9.6.1 Modelli di database

Nei database distinguiamo due aspetti:

1. **Modello** (o **schema**): la *descrizione* dei dati e le *relazioni* tra le varie parti del dataset
2. **Istanza** (o **stato**): il contenuto vero e proprio nel database, cioè i *dati*

Ciò su cui focalizziamo la nostra attenzione per il momento è proprio il modello, ovvero "l'ossatura" del database. È evidente che i dati di tipo *strutturato* si adattano meglio a un'organizzazione schematica, dunque per essi si offriranno dei tipi di database certamente differenti da quelli offerti per dati semi-strutturati e non strutturati. Tale schema è fornito direttamente dal DBMS, ed è considerato l'aspetto più rilevante che differenzia i vari tipi di database tra loro.<sup>7</sup>

Uno dei modi più usati dai modelli per permettere l'accesso a un record è attraverso l'uso delle **chiavi**: affinché ogni record sia *raggiungibile*, infatti, può essere pensato come un valore indicizzato da un identificatore, che è appunto la chiave. Questo è il metodo utilizzato dai **database relazionali**, ovvero il modello di database più diffuso e considerato lo standard per i dati strutturati; è utilizzato anche nei *database key-value*, adatti per dati semi-strutturati e non strutturati, dove ogni record è identificato da una coppia valore-chiave univoca.

In generale, comunque, distinguiamo due tipi di modelli: *relazionali* e *non relazionali*; i primi, come detto, sono lo standard per i dati strutturati, e sono anche chiamati *SQL database* a causa del linguaggio informatico più utilizzato per lavorare con essi; i secondi, invece, sono tipicamente realizzati per dati di tipo semi-strutturato e non strutturato, e ci si riferisce ad essi anche con termine *NoSQL*, che sta per "Not Only SQL", nel senso che i dati non devono necessariamente essere di tipo strutturato.

---

<sup>7</sup>Altri aspetti che differenziano i database sono, ad esempio, il tipo di accesso (a singolo utente o multiutente) e l'architettura (centralizzata in un unico database o federata su più database autonomi).

### 9.6.2 Database relazionali

I primi software che si occupassero della gestione di database relazionali vennero ideati negli anni '70 grazie all'**algebra relazionale** introdotta dall'informatico Ted Codd in quegli anni. Il più utilizzato oggi è *MySQL* trattandosi di un management system open source.

Come già affermato sopra, i database relazionali richiedono uno schema predefinito e noto a priori, dunque si adatta perfettamente ai dati strutturati ma non agli altri tipi. Non a caso, le collezioni dei RDB (*Relational DataBase*) sono delle **tabelle** organizzate in colonne (definibili come categorie) e righe (i singoli record distinti tra loro); questa struttura, come vedremo di seguito, facilita la *mappatura* tra tabelle diverse, ovvero le relazioni che si instaurano tra di esse.

Vediamo un esempio di database contenente informazioni su delle moto con tre record:

| bike_id | model      | model_id |
|---------|------------|----------|
| 100     | Goldwing   | 999      |
| 101     | AfricaTwin | 12       |
| 102     | CBR1000    | 455      |

Tabella 9.1: Esempio di tabella in un database relazionale

Distinguiamo chiaramente tre elementi:

- I *record* corrispondenti alle singole righe della tabella, dette anche *tuple*
- Gli **attributi** corrispondenti alle singole colonne
- La **primary key** (*chiave primaria*) corrispondente a un attributo specifico;<sup>8</sup> essa deve rispondere a due caratteristiche fondamentali: deve essere **unica** (nel senso che tuple differenti non possono avere key uguali) e deve essere **minimale** (nel senso che è richiesto il numero minimo di attributi per definire tale key)

Le singole tabelle permettono di ordinare i record elencando tutte le informazioni secondo la primary key. Per legare le varie tabelle viene fatto uso delle cosiddette **foreign key** (*chiavi esterne*), ovvero attributi o combinazioni di attributi in delle tabelle che fungono però da primary key in un'altra tabella. Riprendendo l'esempio appena visto in Tabella 9.1 ne vediamo un'ampliamento in Tabella 9.2 con più tabelle legate dalle foreign key.

<sup>8</sup>In generale, può corrispondere anche a una *combinazione* di più attributi.

| <i>bike_id</i> | model      | <i>model_id</i> | <i>model_id</i> | year | <i>engine_id</i> | <i>chassis_id</i> |
|----------------|------------|-----------------|-----------------|------|------------------|-------------------|
| 100            | Goldwing   | 999             | 12              | 2016 | 82               | 97                |
| 101            | AfricaTwin | 12              | 75              | 1989 | 56               | 90                |
| 102            | CBR1000    | 455             | 86              | 2011 | 83               | 112               |

| <i>engine_id</i> | engine name | n_cilinders | n_valves | <i>valve type</i> | <i>valve_id</i> | <i>part_n</i> |
|------------------|-------------|-------------|----------|-------------------|-----------------|---------------|
| 81               | 699F        | 4           | 3        | 128               | 96              | vlv_1256      |
| 82               | 1000L       | 2           | 4        | 96                | 97              | vlv_1286      |
| 83               | 350X        | 1           | 2        | 622               | 98              | vlv_1193      |

Tabella 9.2: Esempio di relazioni tra tabelle in uno stesso database; in corsivo è riportata la stessa tupla nelle varie tabelle

In questo caso specifico abbiamo quattro tabelle, in ciascuna delle quali è presente una primary key (ovviamente) e una foreign key; in generale una tabella può presentare più foreign key: ad esempio, nella tabella identificata dalla chiave *model\_id* l'attributo *chassis\_id* potrebbe essere una chiave esterna per un'ulteriore tabella.

Ogni insieme di operazioni eseguite su un database relazionale è chiamata **transaction**, e ciascuna di esse deve rispettare quattro criteri riassunti nell'acronimo **ACID**:

- **Atomicity**: detto anche *all-or-nothing*, indica che tutte le operazioni in una transaction o sono eseguite in forma *completa* oppure l'intera transaction viene *abortita*; in altre parole, non è possibile che il database subisca variazioni a causa di solo alcune operazioni e non altre nella stessa transaction.
- **Consistency**: ogni transaction deve portare un database da uno *stato valido* a un *altro stato valido*. Per “stato valido” intendiamo uno stato in cui siano conservati tutti i vincoli imposti, dunque se - ad esempio - un attributo è associato a una stringa da *n* caratteri fissati ciò deve valere per qualunque transaction coinvolga la tabella in cui è contenuto tale attributo.<sup>9</sup>
- **Isolation**: ogni transaction su un database a cui hanno accesso più utenti deve generare gli stessi risultati che sarebbero stati ottenuti in un ambiente a singolo utente per evitare conflitti. Ciò è molto importante nel momento in cui avvengono transaction diverse da parte di utenti diversi in tempi molto vicini tra loro: l'indipendenza di una transaction da un'altra farà sì che la coerenza globale nei risultati di tutti gli utenti sia conservata.
- **Durability**: in seguito a una transaction lo stato del database deve rimanere lo stesso fino a quando non viene modificato da un'altra transaction.

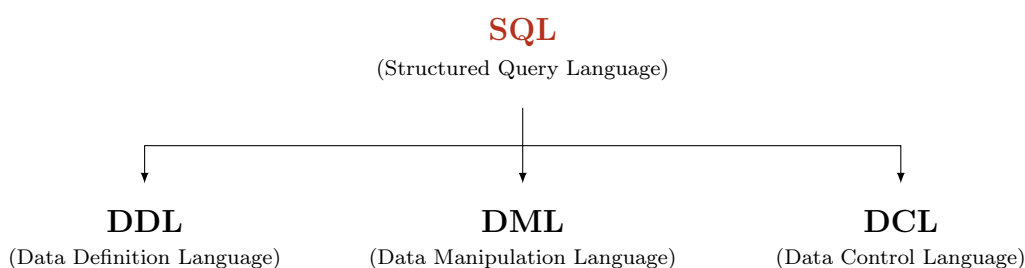
<sup>9</sup>Com'è evidente dalla spiegazione data, questo concetto di *consistency* non ha nulla a che vedere con quello visto nel teorema CAP nonostante l'omonimia.

### 9.6.3 SQL

Il linguaggio standard per ogni DBMS legato ai database relazionali è l'**SQL**, che può essere pronunciato sia come acronimo sia come il termine *sequel*, in quanto originariamente era chiamato *SEQUEL* (*Structured English Query Language*) e solo in un secondo momento vennero rimosse le vocali per motivi di conflitto con marchi registrati.

Si tratta di un linguaggio ormai datato, essendo il suo sviluppo iniziato nel 1974 e divenuto standard dal 1986, anno in cui venne adottato dall'ISO (organizzazione globale più importante per la definizione di norme tecniche).

Tale linguaggio, inoltre, è di tipo **dichiarativo** (*declarative language*), nel senso che quanto viene scritto è relativo principalmente alla descrizione di quanto deve essere fatto, piuttosto che sul come debba essere fatto. Esso può essere strutturato in tre parti:



Vedremo di seguito i vari comandi utili a lavorare con i database.

#### Creazione di database e tabelle

Ogni istruzione in SQL deve terminare con il punto e virgola.

Per visualizzare i database presenti in memoria è possibile usare il seguente comando:

```
SHOW DATABASES;
```

da cui segue un elenco dei database.

Per creare un database il comando è **CREATE** seguito dal nome del database; per evitare problemi è sempre meglio specificare che la creazione è possibile solo se un database con quel nome non esiste già:

```
CREATE DATABASE IF NOT EXISTS MyDataBase;
```

Per lavorare con un database specifico è necessario scrivere **USE** seguito dal nome del database:

```
USE MyDataBase;
```

Usando un database possiamo farci mostrare le tabelle contenute in esso:

```
SHOW TABLES;
```

Per creare una tabella dovremo specificare i nomi degli attributi insieme al tipo di variabile associata. I tipi di variabili in SQL sono diverse; alcuni esempi sono mostrati in tabella 9.3.

| Tipo di dato             | Descrizione                                                                 | Limite                  | Default            |
|--------------------------|-----------------------------------------------------------------------------|-------------------------|--------------------|
| <code>char(n)</code>     | Stringa di lunghezza $n$ fissata                                            | $n = 2^8$               | $n = 1$            |
| <code>varchar(n)</code>  | Stringa di lunghezza variabile massima $n$                                  | $n = 2^{16}$            |                    |
| <code>boolean</code>     | Booleano                                                                    | $\{0, 1\}$              |                    |
| <code>smallint</code>    | Numero intero                                                               | $[-2^{15}, 2^{15} - 1]$ |                    |
| <code>int</code>         | Numero intero                                                               | $[-2^{31}, 2^{31} - 1]$ |                    |
| <code>float(n,d)</code>  | Numero in virgola mobile con $n$ cifre massime e $d$ cifre decimali massime | $(n, d) = (24, 23)$     | $(n, d) = (10, 2)$ |
| <code>double(n,d)</code> | Numero in virgola mobile con $n$ cifre massime e $d$ cifre decimali massime | $(n, d) = (53, 52)$     | $(n, d) = (16, 4)$ |
| <code>date</code>        | Data in formato YYYY-MM-DD                                                  |                         |                    |
| <code>time</code>        | Tempo in formato hh-mm-ss                                                   |                         |                    |
| <code>datetime</code>    | Data e tempo in formato YYYY-MM-DD hh-mm-ss                                 |                         |                    |

Tabella 9.3: Principali tipi di dati in SQL

Vediamo un esempio di creazione di tabella:

```
CREATE TABLE Users (
    UserID    varchar(30),
    BadgeNum  int,
    FirstName varchar(255),
    LastName  varchar(255),
    Age       int,
    OtherAttr float
);
```

## Chiavi

Per specificare la primary key è necessario scrivere l'opzione `PRIMARY KEY` accanto all'attributo desiderato:

```
CREATE TABLE Users (
    UserID    varchar(30) PRIMARY KEY,
    BadgeNum  int,
    FirstName varchar(255),
    LastName  varchar(255),
    Age       int,
    OtherAttr float
);
```



Se invece la chiave è data da un insieme di attributi non bisogna scrivere **PRIMARY KEY** accanto ciascun attributo, perché SQL interpreterebbe come se venissero posti gli attributi singolarmente come chiavi primarie e restituirà il seguente errore:

```
CREATE TABLE Users (  
    UserID    varchar(30)    PRIMARY KEY,  
    BadgeNum  int            PRIMARY KEY,  
    FirstName varchar(255),  
    LastName  varchar(255),  
    Age       int,  
    OtherAttr float  
);
```

```
ERROR 1068 (42000): Multiple primary key defined
```

Per definire una primary key con più attributi bisogna scrivere un'ulteriore riga in cui vengono specificati tra parentesi gli attributi da prendere:

```
CREATE TABLE Users (  
    UserID    varchar(30),  
    BadgeNum  int,  
    FirstName varchar(255),  
    LastName  varchar(255),  
    Age       int,  
    OtherAttr float,  
    PRIMARY KEY (UserID,BadgeNum)  
);
```

Per indicare una foreign key, invece, è necessario specificare tra parentesi l'attributo della tabella e usare l'istruzione **REFERENCES** con oggetto il nome della tabella in cui la chiave è primaria seguita, tra parentesi, dal nome della chiave primaria stessa.

```
CREATE TABLE Authorizations (  
    BadgeID    int PRIMARY KEY,  
    Since      date,  
    Level      int,  
    Sector     int  
);  
  
CREATE TABLE Users (  
    UserID      varchar(30) PRIMARY KEY,  
    BadgeNum    int,  
    FirstName   varchar(255),  
    LastName    varchar(255),  
    Age         int,  
    OtherAttr   float,  
    FOREIGN KEY (BadgeNum) REFERENCES Authorizations(BadgeID)  
);
```

## Vincoli

A ogni attributo possono essere associati uno o più vincoli, indicati da parole chiave in maiuscolo che vanno riportate dopo il tipo di variabile associata all'attributo.

Uno dei possibili vincoli è la richiesta che ogni record associato all'attributo sia *unico*; ad esempio, quando viene definita una foreign key è tipico che essa coincida con un attributo unico. La parola chiave in questo caso è **UNIQUE**.

Un secondo vincolo è la richiesta che in un attributo non possano esservi valori detti **NULL**, che sono presenti quando in un record non viene specificato suddetto attributo. Per imporlo basta aggiungere il comando **NOT NULL**.

Sulla scia del precedente vincolo, è possibile impostare un valore di default quando non specificato dall'utente nell'inserimento dei record; ciò si ottiene tramite il comando **DEFAULT** seguito dal valore di default da impostare, che deve ovviamente essere coerente con la tipologia di variabile indicata.

In ultimo, è possibile fissare una condizione sui valori dell'attributo in modo che venga restituito errore se tale condizione non venga rispettata. Le condizioni vengono sempre poste tra parentesi tonde, e in questo caso ciascuna va preceduta dal comando **CHECK**.

Vediamo un esempio di applicazione di questi vincoli alla tabella `Users`:

```
CREATE TABLE Users (
  UserID      varchar(30) PRIMARY KEY,
  BadgeNum    int         NOT NULL UNIQUE,
  FirstName   varchar(255) NOT NULL DEFAULT '',
  LastName    varchar(255) NOT NULL DEFAULT '',
  Age         int         DEFAULT 18,
  OtherAttr   float       CHECK (OtherAttr>0)
);
```

Per osservare una tabella in modo che vengano esplicitati gli attributi e le caratteristiche ad essi associati è possibile usare il comando `DESCRIBE` seguito dal nome della tabella.

```
DESCRIBE Users;
```

| Field     | Type         | Null | Key | Default | Extra |
|-----------|--------------|------|-----|---------|-------|
| UserID    | varchar(30)  | NO   | PRI | NULL    |       |
| BadgeNum  | int          | NO   | UNI | NULL    |       |
| FirstName | varchar(255) | NO   |     |         |       |
| LastName  | varchar(255) | NO   |     |         |       |
| Age       | int          | YES  |     | 18      |       |
| OtherAttr | float        | YES  |     | NULL    |       |

### Alterazione delle tabelle

Una volta che lo schema della tabella è stato creato può venire modificato, tenendo sempre presente le varie restrizioni imposte tramite vincoli. Si tenga presente che l'alterazione della struttura di una tabella coinvolge tutti i record in essa presenti: non a caso, uno dei problemi principali nel lavorare con database relazionali è quello di mantenerli coerenti; per questo motivo è stato dato un termine specifico alla creazione e mantenimento dei RDB, ovvero ***database normalization***.

Ogni alterazione di tabella viene eseguita tramite l'incipit `ALTER TABLE` seguito dal nome della tabella.

Per aggiungere una colonna si scrive il comando `ADD COLUMN` seguito dal nome della colonna, dal tipo di variabile associata e da eventuali vincoli dell'attributo:

```
ALTER TABLE Users
ADD COLUMN Role char(1) DEFAULT 'A' CHECK
(Role IN ('A','B','C'));
```

Per impostare un valore di default a un attributo già esistente (o per reimpostarlo nel caso ne avesse già uno) usiamo una sintassi che prevede i nuovi comandi ALTER e SET DEFAULT:

```
ALTER TABLE Users
ALTER Role SET DEFAULT 'B';
```

Per aggiungere un vincolo a un attributo già esiste è possibile utilizzare il comando MODIFY; in SQL modificare un attributo implica a tutti gli effetti ridefinirlo, dunque è necessario scrivere - accanto al nome della colonna - il tipo di variabile:

```
ALTER TABLE Users
MODIFY Role char(1) NOT NULL;
```

Se è stata creata una tabella senza specificare la primary key è possibile farlo in seguito con la seguente sintassi:

```
CREATE TABLE AddressBook (
    TelephoneNumber char(10),
    User            varchar(30)
);
```

```
ALTER TABLE AddressBook
ADD PRIMARY KEY (TelephoneNumber,User);
```

Per definire una foreign key, invece, la sintassi è un po' più complessa:

```
ALTER TABLE AddressBook
ADD CONSTRAINT FK_user
FOREIGN KEY (User) REFERENCES Users(UserID);
```

FK\_user non è altro che il nome del vincolo fornito direttamente dall'utente, che può anche essere rimosso con la seguente sintassi:

```
ALTER TABLE AddressBook
DROP CONSTRAINT FK_user;
```

Nel caso in cui non ci si ricordi del nome dato è possibile guardare tutte le informazioni di una tabella dalla sua creazione tramite il comando SHOW CREATE TABLE:

```
SHOW CREATE TABLE AddressBook;
```

Vediamo ora come eliminare un attributo:

```
ALTER TABLE Users
DROP COLUMN Role;
```

Per eliminare un'intera tabella sarà sufficiente scrivere

```
DROP TABLE IF EXISTS Users;
```

L'eliminazione di una tabella implica non solo la cancellazione di tutti i record in essa contenuti, ma anche di tutte le eventuali connessioni ad altre tabelle tramite foreign key, rendendo potenzialmente inutilizzabile l'intero database, dunque si raccomanda prudenza nell'utilizzo di questo comando.

### Inserimento dati

Per ogni inserimento di record è necessario specificare gli attributi associati alla tupla:

```
INSERT INTO Users (UserID, BadgeNum, FirstName, LastName,  
                  OtherAttr)  
VALUES ('usr:00001', 100, 'Rei', 'Ayanami', 1.8);
```

Separando le tuple tramite virgole è possibile inserire più record in una volta sola:

```
INSERT INTO Users (UserID, BadgeNum, FirstName, LastName,  
                  OtherAttr)  
VALUES  
    ('usr:00002', 101, 'Asuka', 'Soryu Langley', 3.5),  
    ('usr:00003', 102, 'Shinji', 'Ikari', 2.9),  
    ('usr:00004', 103, 'Toji', 'Suzuhara', 4.7),  
    ('usr:00005', 104, 'Kaworu', 'Nagisa', 3.0);
```

### Modifica dei record

Il comando che permette di modificare i record in modo individuale o collettivo è UPDATE; a esso seguirà il SET e infine la condizione che stabilisce dove deve essere eseguita tale modifica attraverso il WHERE:

```
UPDATE Users  
SET     Role = 'C'  
WHERE  (LastName = 'Ayanami');
```

È utile definire ora le *wildcard*, ovvero due caratteri speciali utili per estendere le condizioni in cui è valido il WHERE nel caso di stringhe:

- `_` è una *char wildcard*
- `%` è una *string wildcard*

Esse rappresentano un valore qualsiasi che può essere assunto come singolo carattere o come insieme di caratteri, e per essere utilizzati vanno premessi con LIKE. Ad esempio, scrivendo

```
UPDATE Users
SET     Role = 'B'
WHERE  (FirstName LIKE '%o%');
```

per tutti i record al cui attributo `FirstName` corrisponde una stringa con il carattere 'o' sarà imposto l'attributo `Role` pari a 'B'.

Scrivendo invece

```
UPDATE Users
SET     OtherAttr = 5.0
WHERE  (FirstName LIKE '__i%');
```

verrà posto `OtherAttr = 5.0` in tutti i record con `FirstName` pari a una stringa il cui terzo carattere è 'i'.

### Cancellazione dei record

La cancellazione dei record procede allo stesso modo delle modifiche, attraverso il comando `DELETE FROM` seguito dal nome della tabella. Ad esempio, la cancellazione dei record il cui penultimo carattere del `LastName` è 'p' si ha nel seguente modo:

```
DELETE FROM Users
WHERE      (LastName LIKE '%p_');
```

### Selezione dei dati

Il comando `SELECT` è il più potente e utilizzato nel linguaggio SQL, in quanto permette l'estrazione di determinati record secondo delle condizioni specificate dall'utente, ovvero le cosiddette **query**.

La prima utilità di questa funzione è quella della *visualizzazione*; ad esempio, per guardare l'intera tabella è possibile scrivere

```
SELECT * FROM Users;
```

dove '\*' sta a indicare l'intero insieme dei record senza alcuna restrizione.

È possibile imporre un limite ai record visualizzati tramite `LIMIT`:

```
SELECT * FROM Users
LIMIT 3;
```

Per visualizzare solo specifici attributi è possibile specificarli al posto dell'asterisco:

```
SELECT FirstName, BadgeNum FROM Users;
```

Inoltre, è possibile visualizzare tutti gli attributi distinti senza ripetizioni (si pensi ai casi di omonimia); per farlo bisogna accompagnare `DISTINCT` al comando `SELECT`:

```
SELECT DISTINCT FirstName FROM Users;
```

Un'altra possibilità è quella di ordinare la tabella rispetto a un attributo; se non specificato l'ordinamento sarà *ascendente*, ma è possibile avere anche un ordinamento discendente tramite l'opzione `DESC`:

```
SELECT FirstName FROM Users  
ORDER BY FirstName;
```

È possibile specificare altre condizioni tramite `WHERE`:

```
SELECT FirstName, BadgeNum FROM Users  
WHERE (OtherAttr > 0);
```

Un'ulteriore possibilità di visualizzazione consiste nel definire una tabella speciale, chiamata `VIEW`, scegliendo gli attributi e imponendo delle condizioni che devono rispettare i record. Vediamo un esempio:

```
CREATE VIEW OneThree AS  
SELECT      UserID, BadgeNum, LastName  
FROM        Users  
WHERE       (OtherAttr BETWEEN 1 AND 3);
```

`OneThree` apparirà nell'elenco delle tabelle con il comando `SHOW TABLES`, e ne potrà essere visualizzato il contenuto scrivendo

```
SELECT * FROM OneThree;
```

Per osservarne le caratteristiche è possibile scrivere

```
SHOW CREATE VIEW OneThree;
```

Ogni modifica apportata alla tabella principale `Users` verrà riflessa nella visuale `OneThree`, e allo stesso modo è possibile trattare `OneThree` come una tabella a tutti gli effetti - inserendo, modificando ed eliminando record - riflettendo i risultati su `Users`; è anche possibile inserire un record all'interno della visuale che non rispetti i parametri di visualizzazione (in questo caso fornendo un valore di `OtherAttr` al di fuori dell'intervallo `[1, 3]`): in tal caso il record verrà comunque inserito all'interno di `Users`, non figurando comunque in `OneThree`.

Un'altra funzione del **SELECT** permette la visualizzazione di attributi ai cui valori sono stati applicati operazioni matematiche; con la seguente sintassi, ad esempio, vengono riportati tutti i numeri in **OtherAttr** raddoppiati e tutti i numeri in **BadgeNum** elevati al quadrato:

```
SELECT OtherAttr*2, POWER(BadgeNum,2) FROM Users
WHERE (OtherAttr > 1.8);
```

Le operazioni non andranno in questo caso a modificare i valori nella tabella, ma solo quelli visualizzati in output dall'utente.

Un ulteriore strumento per le query sono le funzioni già implementate in SQL; le principali sono le seguenti:

- **COUNT()** restituisce il numero di record associati a una determinata query; ponendo come argomento il singolo attributo, ad esempio, verrà restituito il numero di valori diversi da **NULL** associati a tale attributo
- **MAX()** restituisce il più grande tra un insieme di valori
- **MIN()** restituisce il più piccolo tra un insieme di valori
- **SUM()** restituisce la somma di un insieme di valori
- **AVG()** restituisce la media di un insieme di valori

Per comodità e abbreviazione è possibile definire degli **alias** sia per gli attributi che per intere tabelle, ad esempio:

```
SELECT DISTINCT FirstName AS UniqueNames
FROM Users;
```

In realtà è possibile omettere **AS** e riportare semplicemente il nome accanto all'attributo o tabella che sia:

```
SELECT FirstName NamesEndingI
FROM Users U
WHERE (FirstName LIKE '%i');
```

È possibile raggruppare righe con lo stesso valore per un determinato attributo tramite il **GROUP BY**. Immaginiamo, ad esempio, di avere una tabella di spedizioni denominata **Ships** la cui chiave principale è il codice di spedizione dato da **Code**, le nazioni di partenza delle spedizioni sono date da **FromCountry** e le città di partenza delle medesime spedizioni sono date da **FromCity**, e di voler calcolare quali siano le nazioni con più città da cui partono spedizioni.



La tabella è la seguente:

```
SELECT * FROM Ships;
```

| Code | FromCountry | FromCity | Rate |
|------|-------------|----------|------|
| 001  | Italy       | Rome     | 4.7  |
| 002  | Italy       | Venice   | 3.1  |
| 003  | France      | Nice     | 3.8  |
| 004  | Italy       | Rome     | 2.6  |

Potremo utilizzare la seguente sintassi:

```
SELECT FromCountry, COUNT(DISTINCT FromCity) TotalCities
FROM Ships
GROUP BY FromCountry
ORDER BY TotalCities DESC;
```

| FromCountry | TotaleCities |
|-------------|--------------|
| Italy       | 2            |
| France      | 1            |

Per filtrare i risultati dell'attributo raggruppato attraverso funzioni quali COUNT, SUM e AVG è possibile utilizzare HAVING:

```
SELECT FromCountry, COUNT(DISTINCT FromCity) TotalCities
FROM Ships
GROUP BY FromCountry
HAVING TotalCities>1
ORDER BY TotalCities DESC;
```

| FromCountry | TotaleCities |
|-------------|--------------|
| Italy       | 2            |

L'ultima funzione che vediamo con SELECT è il *join*, ovvero la possibilità di visualizzare delle tabelle unite attraverso dei valori in determinati attributi (tipicamente primary key) uguali per entrambe le tabelle.

Immaginiamo di avere un'altra tabella, chiamata **Orders**, la cui chiave primaria è analoga a quella della tabella **Ships**, e sono presenti i nomi di chi ha effettuato gli ordini (**Name**) e le loro età (**Age**):

```
SELECT * FROM Orders;
```

| Code | Name      | Age |
|------|-----------|-----|
| 002  | Lewis     | 39  |
| 003  | Charles   | 26  |
| 005  | Sebastian | 36  |

La prima possibilità è un **INNER JOIN** in cui vengono mostrati gli attributi richiesti dall'utente dove i valori dell'attributo in comune sono uguali:

```
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
INNER JOIN Orders o ON s.Code = o.Code;
```

| fromCountry | Name    | Rate |
|-------------|---------|------|
| Italy       | Lewis   | 3.1  |
| France      | Charles | 3.8  |

Notiamo una nuova sintassi mai vista prima: gli attributi, infatti, vengono scritti nella forma **NomeTabella.NomeAttributo**; questo è utile nel caso in cui due tabelle condividano lo stesso nome per un attributo, com'è qui il caso per **Code**.

Poiché i codici in comune per le due tabelle sono solo 002 e 003 saranno solo gli attributi di questi due record a essere mostrati. Aggiungiamo anche che l'inner join è - per così dire - *commutativo*, nel senso che invertendo **Ships s** e **Orders o** tra la seconda e la terza stringa di codice avremmo ottenuto lo stesso risultato.

Altri due possibili join sono il **LEFT JOIN** e il **RIGHT JOIN**, che *non sono commutativi*. Vediamo cosa cambia rispetto all'inner join:

```
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
LEFT JOIN Orders o ON s.Code = o.Code;
```

| fromCountry | Name    | Rate |
|-------------|---------|------|
| Italy       | NULL    | 4.7  |
| Italy       | Lewis   | 3.1  |
| France      | Charles | 3.8  |
| Italy       | NULL    | 2.6  |

Vengono quindi presi *tutti* i record della tabella **Ships** e gli attributi relativi ai record non presenti in **Orders** vengono riempiti con **NULL**. Questo avviene perché, secondo quanto abbiamo scritto, è la tabella **Orders** a essere unita alla tabella **Ships**, dunque è quest'ultima a essere selezionata e la prima a essere aggiunta in seguito. Invertendo l'ordine delle tabelle, infatti, il risultato sarà diverso:

```
SELECT s.FromCountry, o.Name, s.Rate
FROM Orders o
LEFT JOIN Ships s ON s.Code = o.Code;
```

| fromCountry | Name      | Rate |
|-------------|-----------|------|
| Italy       | Lewis     | 3.1  |
| France      | Charles   | 3.8  |
| NULL        | Sebastian | NULL |

Con il right join avremo il comportamento opposto: in quel caso è la prima tabella citata a essere aggiunta alla seconda.

L'ultimo possibile join è il *full join*, chiamato anche *outer join*, in cui tutti i record vengono aggiunti e i vari attributi non presenti vengono riempiti con **NULL**. In SQL non esiste una funzione esplicita che esegua ciò, ma è possibile farlo unendo tramite **UNION** un left join con un right join.

```
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
LEFT JOIN Orders o ON s.Code = o.Code;
```

UNION

```
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
RIGHT JOIN Orders o ON s.Code = o.Code;
```

| fromCountry | Name      | Rate |
|-------------|-----------|------|
| Italy       | NULL      | 4.7  |
| Italy       | Lewis     | 3.1  |
| France      | Charles   | 3.8  |
| Italy       | NULL      | 2.6  |
| NULL        | Sebastian | NULL |

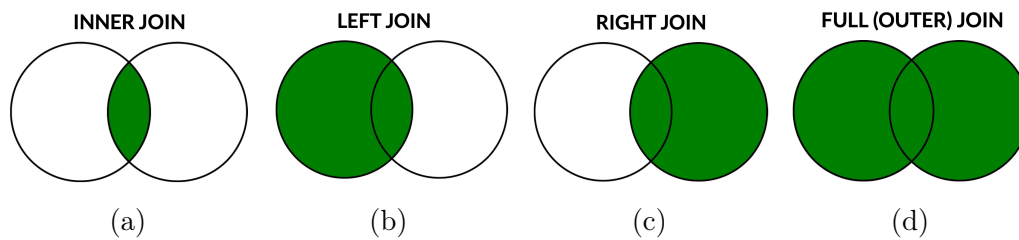


Figura 9.24: Illustrazione grafica dei processi di join

### Controllo dati

L'ultima componente rimasta è la DCL, ovvero la *Data Control Language*, che si occupa dell'accesso ai database, dei permessi concessi agli utenti e della sicurezza dei dati. Non approfondiremo questi aspetti come abbiamo fatto con la DDL e la DML, perché la DCL è meno orientata all'esperienza utente e più all'aspetto di amministrazione, dunque vedremo solo le funzioni più importanti.

La base di partenza è sicuramente quella di creare un nome utente associato a una password:

```
CREATE USER    'new_user'@'localhost'
IDENTIFIED BY 'new_password';
```

Tramite questo comando verrà creato un nuovo utente con nome 'new\_user', che può connettersi unicamente dalla macchina locale ('localhost') e a cui è associata una password 'new\_password'.

È possibile assegnare (tramite **GRANT**) e rimuovere (tramite **REVOKE**) privilegi agli utenti, come la possibilità di usare i comandi **SELECT** e **UPDATE**:

```
GRANT SELECT, UPDATE ON Orders.Name TO 'new_user'@'localhost';
```

```
REVOKE UPDATE ON Orders.Name TO 'new_user'@'localhost';
```

## 9.7 Database NoSQL

Il linguaggio SQL si adatta perfettamente ai database relazioni, predominanti fino agli anni '90 dello scorso secolo, quando la quantità di dati generati dall'esplosione di internet e dalle applicazioni web divenne sempre più crescente. Nel 1998 l'informatico Carlo Strozzi coniò il termine *NoSQL* relizzando un database management system che non richiedesse l'uso del linguaggio SQL, promuovendo l'idea di database open source che si distaccassero dal tradizionale modello relazionale: secondo Strozzi, infatti, quest'ultimo può rappresentare in certi casi una "forzatura" per la quale sarebbe meglio tentare approcci differenti.

### 9.7.1 Partizione dei database

Un primo ostacolo importante nella gestione di un database relazione si presenta al crescere della quantità di record, che obbliga alla **partizione** dei database, ovvero alla loro suddivisione in parti più piccole. Come sappiamo dall'archiviazione dati ci troveremo ad affrontare la scelta tra la partizione verticale e partizione orizzontale: in questo contesto, la prima implica che tutti i record vengano salvati su ogni disco del sistema in modo che siano gli attributi a essere divisi tra un disco e l'altro. Una partizione di questo tipo non è un problema per il modello relazionale, dato che le varie partizioni possono essere viste come tabelle differenti con la stessa chiave primaria.

Il problema sorge con la partizione orizzontale, anche chiamata **sharding**, dove sono i record a essere suddivisi nei diversi dischi in modo che tutti gli attributi associati allo stesso record vengano mantenuti. Questo caso non è previsto dagli RDBMS tradizionali, i quali richiedono che ogni tabella sia interamente inclusa in un unico elemento di memoria; per ovviare a ciò è necessario costruire nuovi tipi di database che possano tenere traccia della distribuzione dei record.

L'idea è quella di considerare la chiave primaria, essendo l'attributo che contraddistingue ogni record da tutti gli altri. Solitamente viene applicata una funzione hash alla primary key che ritorna un numero; con questo viene poi eseguita la divisione intera col numero di partizione disponibili  $P$ : il resto sarà un numero compreso tra 0 e  $P - 1$ , coincidente con l'indice del disco su cui memorizzare il record associato al valore della key.

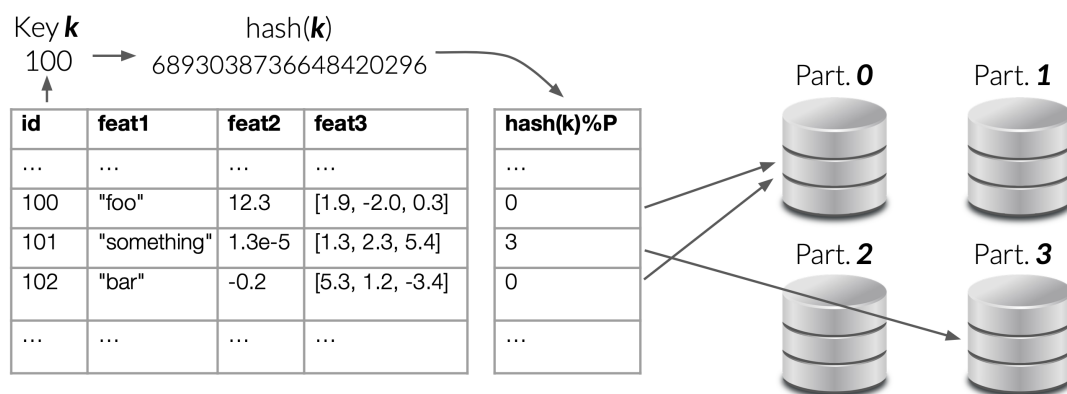


Figura 9.25: Esempio di sharding tramite hashing con divisione intera e  $P = 4$

Questo metodo, però, non risponde bene in termini di scalabilità: aggiungere un nuovo disco, infatti, significa passare da  $P$  a  $P + 1$ , imponendo che tutte le divisioni intere vengano rieseguite e tutti i record spostati secondo la nuova indicizzazione.

Quello che viene fatto nella pratica è il cosiddetto **consistent hashing**, ovvero una mappatura dei digest in un *anello di valori* che va da 0 al numero massimo di digest possibile, che nel caso dello SHA256 è pari a  $2^{256}$ .

Nell'esempio visto sopra la funzione per mappare i digest sarebbe la  $\text{mod}(P)$ , che restituisce appunto la divisione intera con  $P$ . Vediamo cosa succederebbe spostandoci a  $P + 1$  con, ad esempio,  $P = 3$ :

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 0 | 1 | 2 | 0 | 1 | 2 |
| ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ |
| 0 | 1 | 2 | 3 | 0 | 1 | 2 | 3 | 0 |

Essendo impensabile rimappare quasi tutti i record tra i vari dischi si procede in modo diverso: viene associato a ogni disco un “settore” dell’anello, in modo che ogni digest nell’anello venga “arrotondato” al valore più grande all’interno del settore dove avviene l’associazione con la partizione.

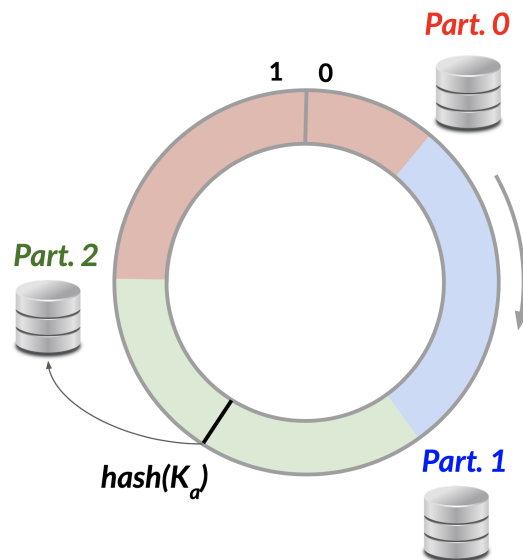


Figura 9.26: Esempio di consistent hashing con tre dischi

Introducendo un nuovo disco quest’ultimo andrà a occupare parte di un settore precedentemente occupato da un altro disco; posizionando il suo dominio tra la partizione 2 e la partizione 0 della figura 9.26, ad esempio, non sarebbe necessario rimappare l’intero database, ma bisognerebbe trasferire solo parte dei record dalla partizione 0 alla nuova partizione 3.

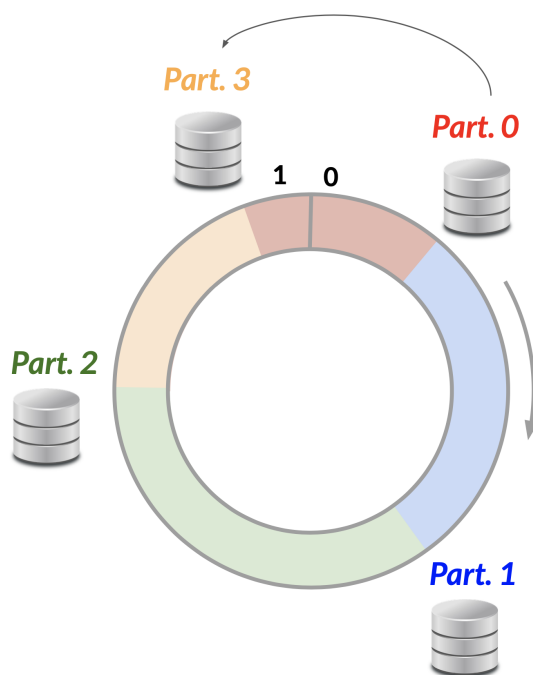


Figura 9.27: Introduzione di una nuova partizione con conseguente trasferimento di record

Il consistent hashing si presta bene anche alla replicazione di dati su più partizioni, in modo da avere più copie e rendere più sicuro il dataset: solitamente l'utente sceglie su quante partizioni distribuire le copie, in base allo spazio che è disposto a sacrificare; scelto il numero  $n$  di partizioni, le copie verranno depositate sulle  $n - 1$  partizioni successive a quella in cui è originariamente ospitato il record.

## 9.8 Caratteristiche dei database NoSQL

I punti che distinguono i database tradizionali da quelli NoSQL sono principalmente quattro:

1. **Modelli flessibili** che possano discostarsi dalla natura strutturata dei database relazionali, offrendo supporto per la memorizzazione e gestione di dati semi-strutturati e non strutturati
2. **Scalabilità orizzontale** di cui abbiamo parlato prima, in modo da rendere possibile una distribuzione dei database su più nodi di un cluster senza un significativo degrado delle prestazioni
3. **Query semplificate** attraverso linguaggi che non fanno uso di operazioni complesse come i JOIN, privilegiando così tempi di risposta più rapidi (soprattutto in ambienti distribuiti) rispetto alla potenza espressiva del linguaggio

4. **Modelli con minore consistency** che aderiscono meno, di conseguenza, alle proprietà ACID, in favore della scalabilità e della *availability* necessarie nei sistemi distribuiti

Il modello di transaction seguito nella maggior parte dei casi dai database NoSQL è il **BASE**, così chiamato per evidenziare l'opposizione all'*ACID* così come avviene in chimica:

- **Basically Available**: i singoli nodi in un network sono generalmente accessibili *quasi* sempre, al contrario del modello ACID in cui l'accesso al servizio potrebbe essere interrotto a causa di un'operazione di lettura o scrittura che sta avvenendo in quel momento per via della *isolation*
- **Soft state**: lo stato di un database non è rigidamente *persistente*, ma esiste la possibilità che esso cambi senza un intervento specifico dell'utente, come nel caso della replicazione dei dati sui vari nodi di un cluster
- **Eventually consistent**: legata al soft state, si intende che la *consistency*<sup>10</sup> è garantita solo oltre un certo tempo, e non necessariamente nel momento esatto in cui viene conclusa una transaction

Specifichiamo che l'ultimo punto non implica che tutti i database NoSQL siano *AP*, dato che non si tratta di una dicotomia in cui viene scelta una proprietà ed esclusa l'altra: sia l'*availability* *A* che la *consistency* *C* vengono garantite in una forma di "gradiente", passando da condizioni più stringenti (*high availability*) a condizioni più labili (*eventual availability*), e questo vale anche per uno stesso DBMS che può così consentire flessibilità ai diversi servizi e casi d'uso.

## 9.9 Esempi di database NoSQL

### 9.9.1 Database key-value

Viene usato il modo più semplice per archiviare dati: si associa a ogni record una *key*, analogamente a quanto viene fatto in python con i dizionari, e così come questi ultimi a ogni *key* può essere associato un tipo di dato diversi, come un numero, una stringa di caratteri, una lista di valori e altro ancora. In sostanza, questo tipo di archiviazione è *schema-less*, non dovendo neanche includere metadati che illustrino la struttura dei dati.

L'assenza di uno schema permette una facile e rapida distribuzione tra i nodi di un network; inoltre, il modello chiave-valore si adatta particolarmente a quei dataset che richiedono un grande numero di letture e scritture in rapida successione, e per questo motivo i dati vengono salvati direttamente sulla RAM per aumentare significativamente le prestazioni in tal senso. Database che gestiscono i dati situati nella memoria principale sono chiamati **in-memory database**.

---

<sup>10</sup>Credo che in questo caso per *consistency* si intenda quella del teorema CAP e non quella del modello ACID.



### 9.9.2 Database document

Invece di salvare coppie chiave-valore, i database document salvano dei *documenti* composti da coppie campo-valore. Si tratta essenzialmente di informazioni codificate in formato JSON che si adattano perfettamente ai dati semi-strutturati, dove i vari documenti possono anche essere organizzati in “collezioni” attraverso un campo - solitamente chiamato *id* - che identifica univocamente il documento stesso, proprio come le primary key nei RDB. Le query vengono eseguite utilizzando i campi dei documenti.

### 9.9.3 Database time series

Sono progettati per memorizzare dati indicizzati da timestamp e per raggrupparli in funzione di finestre temporali. Essi si adattano ai dati empirici prodotti dai sensori e forniscono funzionalità per calcolare metriche e statistiche che aiutino a identificare trend e pattern.

### 9.9.4 FullText Search Engine

Adatti ai dati non strutturati che vengono collezionati come documenti di tipo testuale; proprio sul testo sono incentrate le query che - analogamente a quanto fanno i motori di ricerca come Google - ritornano una “classifica” di documenti sulla base delle corrispondenze con le parole chiave specificate dall’utente.

# Appendice A

## Tabella ASCII

| Decimal | Hex | Char                   | Decimal | Hex | Char    | Decimal | Hex | Char | Decimal | Hex | Char  |
|---------|-----|------------------------|---------|-----|---------|---------|-----|------|---------|-----|-------|
| 0       | 0   | [NULL]                 | 32      | 20  | [SPACE] | 64      | 40  | @    | 96      | 60  | `     |
| 1       | 1   | [START OF HEADING]     | 33      | 21  | !       | 65      | 41  | A    | 97      | 61  | a     |
| 2       | 2   | [START OF TEXT]        | 34      | 22  | "       | 66      | 42  | B    | 98      | 62  | b     |
| 3       | 3   | [END OF TEXT]          | 35      | 23  | #       | 67      | 43  | C    | 99      | 63  | c     |
| 4       | 4   | [END OF TRANSMISSION]  | 36      | 24  | \$      | 68      | 44  | D    | 100     | 64  | d     |
| 5       | 5   | [ENQUIRY]              | 37      | 25  | %       | 69      | 45  | E    | 101     | 65  | e     |
| 6       | 6   | [ACKNOWLEDGE]          | 38      | 26  | &       | 70      | 46  | F    | 102     | 66  | f     |
| 7       | 7   | [BELL]                 | 39      | 27  | '       | 71      | 47  | G    | 103     | 67  | g     |
| 8       | 8   | [BACKSPACE]            | 40      | 28  | (       | 72      | 48  | H    | 104     | 68  | h     |
| 9       | 9   | [HORIZONTAL TAB]       | 41      | 29  | )       | 73      | 49  | I    | 105     | 69  | i     |
| 10      | A   | [LINE FEED]            | 42      | 2A  | *       | 74      | 4A  | J    | 106     | 6A  | j     |
| 11      | B   | [VERTICAL TAB]         | 43      | 2B  | +       | 75      | 4B  | K    | 107     | 6B  | k     |
| 12      | C   | [FORM FEED]            | 44      | 2C  | ,       | 76      | 4C  | L    | 108     | 6C  | l     |
| 13      | D   | [CARRIAGE RETURN]      | 45      | 2D  | -       | 77      | 4D  | M    | 109     | 6D  | m     |
| 14      | E   | [SHIFT OUT]            | 46      | 2E  | .       | 78      | 4E  | N    | 110     | 6E  | n     |
| 15      | F   | [SHIFT IN]             | 47      | 2F  | /       | 79      | 4F  | O    | 111     | 6F  | o     |
| 16      | 10  | [DATA LINK ESCAPE]     | 48      | 30  | 0       | 80      | 50  | P    | 112     | 70  | p     |
| 17      | 11  | [DEVICE CONTROL 1]     | 49      | 31  | 1       | 81      | 51  | Q    | 113     | 71  | q     |
| 18      | 12  | [DEVICE CONTROL 2]     | 50      | 32  | 2       | 82      | 52  | R    | 114     | 72  | r     |
| 19      | 13  | [DEVICE CONTROL 3]     | 51      | 33  | 3       | 83      | 53  | S    | 115     | 73  | s     |
| 20      | 14  | [DEVICE CONTROL 4]     | 52      | 34  | 4       | 84      | 54  | T    | 116     | 74  | t     |
| 21      | 15  | [NEGATIVE ACKNOWLEDGE] | 53      | 35  | 5       | 85      | 55  | U    | 117     | 75  | u     |
| 22      | 16  | [SYNCHRONOUS IDLE]     | 54      | 36  | 6       | 86      | 56  | V    | 118     | 76  | v     |
| 23      | 17  | [END OF TRANS. BLOCK]  | 55      | 37  | 7       | 87      | 57  | W    | 119     | 77  | w     |
| 24      | 18  | [CANCEL]               | 56      | 38  | 8       | 88      | 58  | X    | 120     | 78  | x     |
| 25      | 19  | [END OF MEDIUM]        | 57      | 39  | 9       | 89      | 59  | Y    | 121     | 79  | y     |
| 26      | 1A  | [SUBSTITUTE]           | 58      | 3A  | :       | 90      | 5A  | Z    | 122     | 7A  | z     |
| 27      | 1B  | [ESCAPE]               | 59      | 3B  | ;       | 91      | 5B  | [    | 123     | 7B  | {     |
| 28      | 1C  | [FILE SEPARATOR]       | 60      | 3C  | <       | 92      | 5C  | \    | 124     | 7C  |       |
| 29      | 1D  | [GROUP SEPARATOR]      | 61      | 3D  | =       | 93      | 5D  | ]    | 125     | 7D  | }     |
| 30      | 1E  | [RECORD SEPARATOR]     | 62      | 3E  | >       | 94      | 5E  | ^    | 126     | 7E  | ~     |
| 31      | 1F  | [UNIT SEPARATOR]       | 63      | 3F  | ?       | 95      | 5F  | _    | 127     | 7F  | [DEL] |

# Appendice B

## Tabellina del 16

### 16 TIMES TABLE

CHART

|               |               |
|---------------|---------------|
| 1 x 16 = 16   | 11 x 16 = 176 |
| 2 x 16 = 32   | 12 x 16 = 192 |
| 3 x 16 = 48   | 13 x 16 = 208 |
| 4 x 16 = 64   | 14 x 16 = 224 |
| 5 x 16 = 80   | 15 x 16 = 240 |
| 6 x 16 = 96   | 16 x 16 = 256 |
| 7 x 16 = 112  | 17 x 16 = 272 |
| 8 x 16 = 128  | 18 x 16 = 288 |
| 9 x 16 = 144  | 19 x 16 = 304 |
| 10 x 16 = 160 | 20 x 16 = 320 |

# Appendice C

## Calcolo del throughput nel RAID 0

Sia  $n$  il numero di volumi utilizzati per memorizzare una quantità di dati  $D$  coinvolta in un'operazione I/O, e assumiamo per semplicità che essi siano equamente distribuiti attraverso le  $n$  unità, in modo che ognuna di esse contenga esattamente  $\frac{D}{n}$  dati.

Il throughput medio  $\tau_m$  sarà dato dal rapporto tra la memoria totale  $D$  occupata dai dati e dalla somma dei tempi con cui i singoli dischi eseguono un'operazione di lettura/scrittura:

$$\tau_m = \frac{D}{\sum_{i=1}^n t_i}$$

Il tempo  $t_i$  è dato dal rapporto tra la quantità di dati in un disco, ovvero  $\frac{D}{n}$ , e il throughput del relativo disco:

$$t_i = \frac{D}{n\tau_i}$$

Da cui segue

$$\tau_m = \frac{\cancel{D}}{\cancel{\frac{D}{n}} \sum_{i=1}^n \frac{1}{\tau_i}} = \frac{n}{\frac{1}{\tau_1} + \frac{1}{\tau_2} + \dots + \frac{1}{\tau_n}} = \frac{n\tau_1\tau_2 \dots \tau_n}{\tau_2\tau_3 \dots \tau_n + \tau_1\tau_3 \dots \tau_n + \dots + \tau_1\tau_2 \dots \tau_{n-1}}$$

Che espressa in modo compatto sarà

$$\tau_m = \frac{n \prod_{i=1}^n \tau_i}{\sum_{i=1}^n \left( \prod_{\substack{j=0 \\ j \neq i}}^n \tau_j \right)} \quad \Rightarrow \quad \begin{array}{ll} n=2 & \rightarrow \tau_m = \frac{n\tau_1\tau_2}{\tau_1 + \tau_2} \\ n=3 & \rightarrow \tau_m = \frac{n\tau_1\tau_2\tau_3}{\tau_1\tau_2 + \tau_1\tau_3 + \tau_2\tau_3} \\ & \dots \end{array}$$